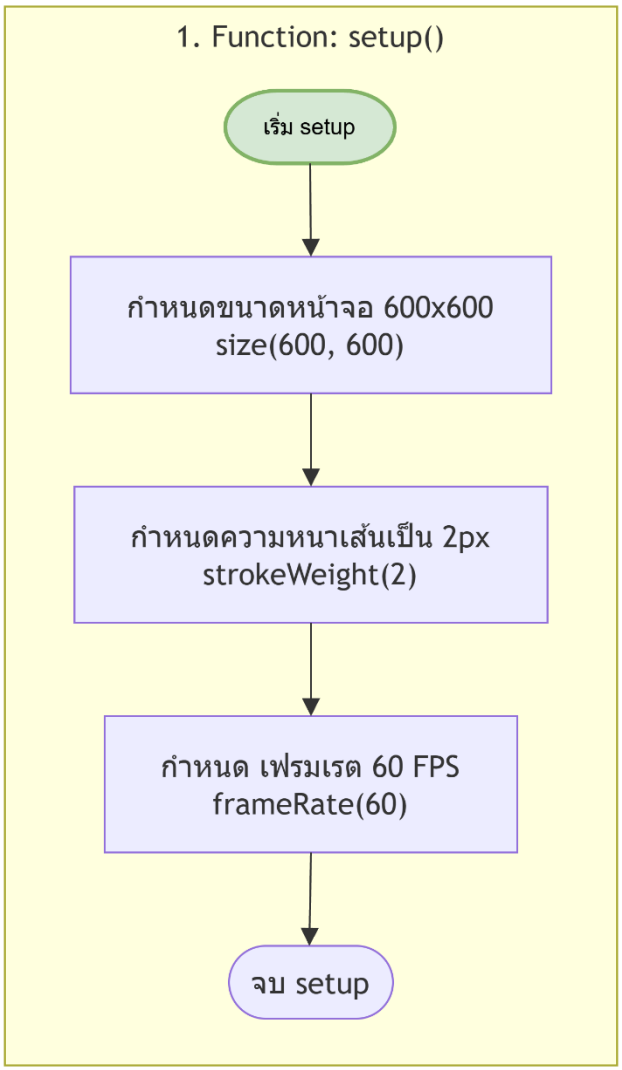
Graphic Mode

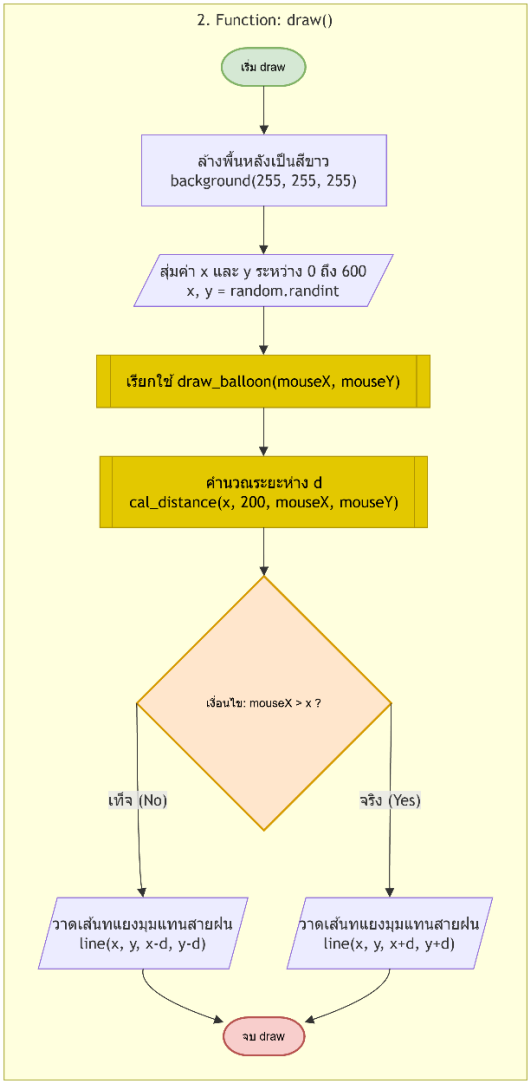
เครื่องมือที่ใช้ : Processing V4.0a4

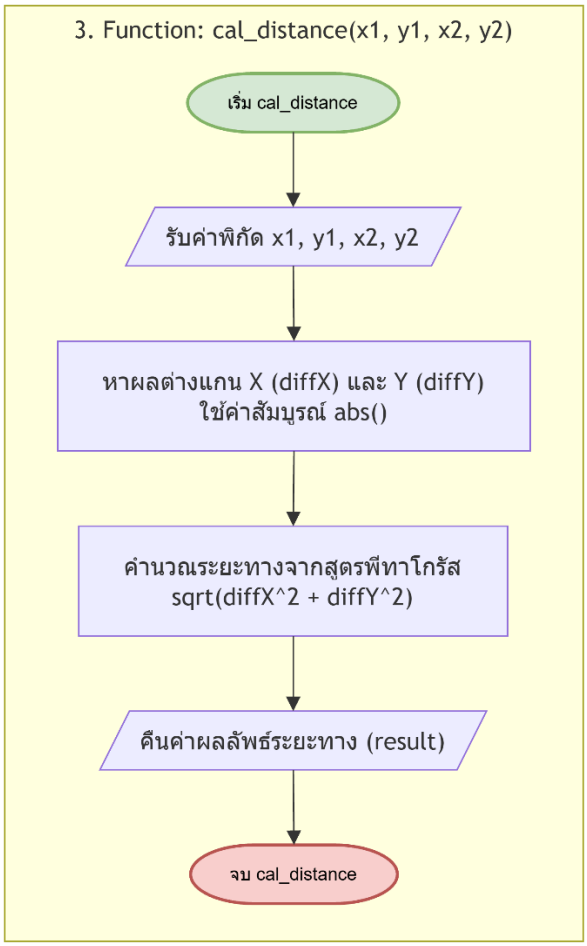
ข้อที่ 1

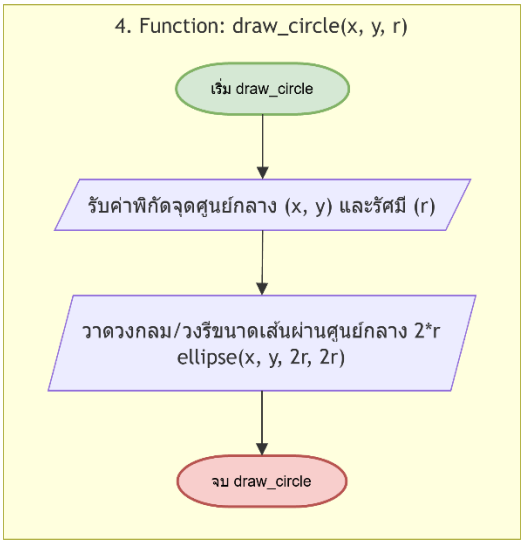
ทำการทดสอบตามในสไลด์หัวข้อ Raining โดยจากในสไลด์กระบวนการแสดงลูกโป่งเป็นไปตามปกติ แต่จะมีข้อผิดพลาดที่การทำสายฝน โดยหากอิงตามสไลด์สายฝนที่เกิดขึ้นจะเป็นแนวนอน เพราะการเรียกใช้ฟังก์ชัน cal_distance() ที่เก็บไว้ในตัวแปร d เมื่อนำไปวาดเส้นด้วยฟังก์ชัน line() จากสไลด์ได้นำ d ไปบวกกับค่าของพิกัด x ทำให้เกิดการวาดเส้นในแนวนอนแทนที่จะวาดเส้นสายฝนในแนวตั้ง โดยผู้ศึกษาได้ทำการนำค่า d ไปคำนวณพิกัดจุดปลายของทั้งแกน x และ y เพื่อให้สายฝนเป็นแนวทแยง

Flowchart






3. Function: cal_distance(x1, y1, x2, y2)
เริ่ม cal_distance
รับค่าพิกัด x1, y1, x2, y2
หาผลต่างแกน X (diffX) และ Y (diffY)
ใช้ค่าสัมบูรณ์ abs()
คำนวณระยะทางจากสูตรพีทาโกรัส
sqrt(diffX^2 + diffY^2)
คืนค่าผลลัพธ์ระยะทาง (result)
จบ cal_distance


4. Function: draw_circle(x, y, r)
เริ่ม draw_circle
รับค่าพิกัดจุดศูนย์กลาง (x, y) และรัศมี (r)
วาดวงกลม/วงรีขนาดเส้นผ่านศูนย์กลาง 2*r
ellipse(x, y, 2r, 2r)
จบ draw_circle

Code

```
import random


def setup():
    size(600, 600)
    strokeWeight(2)
    frameRate(60)


def draw():
    background(255, 255, 255)

    x = random.randint(0, 600)
    y = random.randint(0, 600)

    draw_balloon(mouseX, mouseY)

    d = cal_distance(x, 200, mouseX, mouseY)

    if mouseX > x:
        line(x, y, x + d, y + d)
    else:
        line(x, y, x - d, y - d)

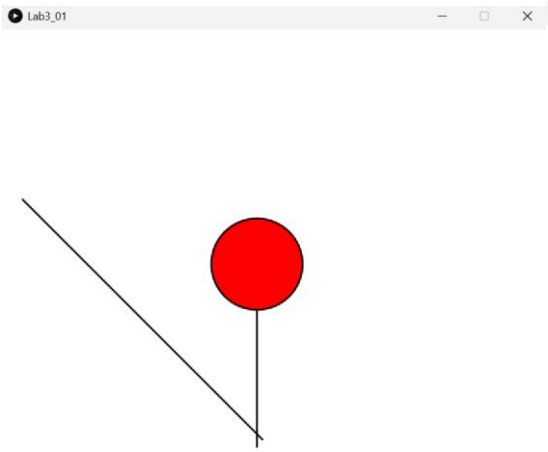


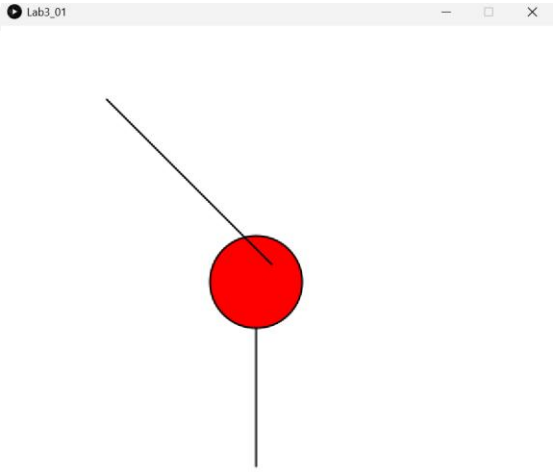
def cal_distance(x1, y1, x2, y2):
    diffX = abs(x1 - x2)
    diffY = abs(y1 - y2)

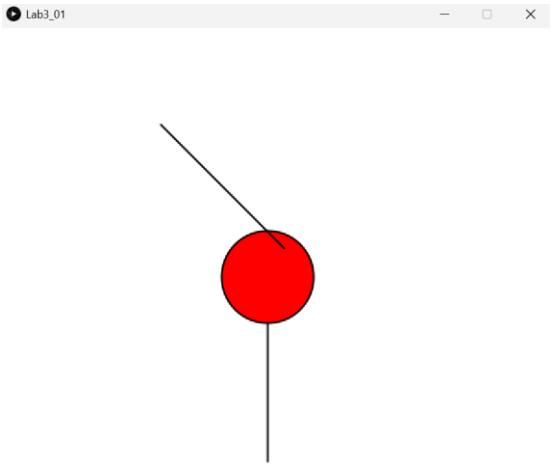
    result = sqrt(diffX * diffX + diffY * diffY)

    return result
def draw_circle(x, y, r):
    ellipse(x, y, 2 * r, 2 * r)


def draw_balloon(x, y):
    fill(255, 0, 0)
    draw_circle(x, y, 50)
    line(x, y + 50, x, y + 200)
```


Lab3_01


Lab3_01


Lab3_01

ข้อที่ 2

โปรแกรมนี้เป็นภาพเคลื่อนไหวเชิงโต้ตอบ (Interactive Animation) ที่แสดงผลรูป เครื่องหมายบวก ซึ่งประกอบขึ้นมาจากสี่เหลี่ยมจัตุรัส 5 รูปต่อกัน ตัวเครื่องหมายบวกหลักจะย่อ-ขยายขนาดตามจังหวะเวลาอย่างต่อเนื่อง และเคลื่อนที่ตามพิกัดเมาส์ของผู้ใช้อย่างนุ่มนวล โดยผู้ใช้สามารถกดคลิกเมาส์ปุ่มซ้ายเพื่อ วางรูปเครื่องหมายบวก ณ ขนาดและตำแหน่งนั้นๆ

2. การกำหนดตัวแปรและการตั้งค่าเริ่มต้น

- **ตัวแปรระดับโกลบอล (Global Variables):**
    - กำหนดพิกัดเริ่มต้นของเครื่องหมายบวกที่ตำแหน่งศูนย์กลาง
    - กำหนดขนาดพื้นฐานของสี่เหลี่ยมจัตุรัส และค่าสเกลเริ่มต้น
    - กำหนดตัวแปรสถานะแบบบูลีนเพื่อควบคุมทิศทางการย่อหรือขยายตัว
    - สร้างรายการ (List) สำหรับเก็บข้อมูลตำแหน่งและขนาดของรูปทรงที่ถูกประทับลงบนหน้าจอ
- **ฟังก์ชันการตั้งค่า (Setup):**
    - กำหนดขนาดของผืนผ้าใบเป็น 1000 x 600 พิกเซล
    - กำหนดโหมดการวาดรูปสี่เหลี่ยมให้ใช้อ้างอิงจากจุดศูนย์กลาง
    - ตั้งค่ารูปแบบเส้นขอบเป็นสีฟ้าโทนเหล็ก (Steel Blue) ความหนา 1.5 พิกเซล และตั้งค่าไม่ให้ใส่สีพื้นหลังภายในรูปทรง (มีเพียงเส้นขอบ)

3. ฟังก์ชันสำหรับสร้างรูปทรง

- **ฟังก์ชันวาดสี่เหลี่ยมจัตุรัสเดี่ยว:** ทำหน้าที่รับพิกัดแกน X, Y และขนาด เพื่อวาดรูปสี่เหลี่ยมจัตุรัส 1 รูปตามตำแหน่งที่ระบุ
- **ฟังก์ชันวาดเครื่องหมายบวก:** ทำหน้าที่สร้างรูปทรงเครื่องหมายบวกโดยการเรียกใช้ฟังก์ชันวาดสี่เหลี่ยมจัตุรัสเดี่ยวซ้ำกัน 5 ครั้ง ได้แก่ สี่เหลี่ยมตรงจุดศูนย์กลาง และสี่เหลี่ยมอีก 4 รูปที่ถูกจัดวางถัดออกไปในทิศทาง บน, ล่าง, ซ้าย และ ขวา

4. กลไกการทำงานของภาพเคลื่อนไหว

ในฟังก์ชันการทำงานหลักที่รันวนลูปอย่างต่อเนื่อง มีขั้นตอนการประมวลผลตามลำดับดังนี้:

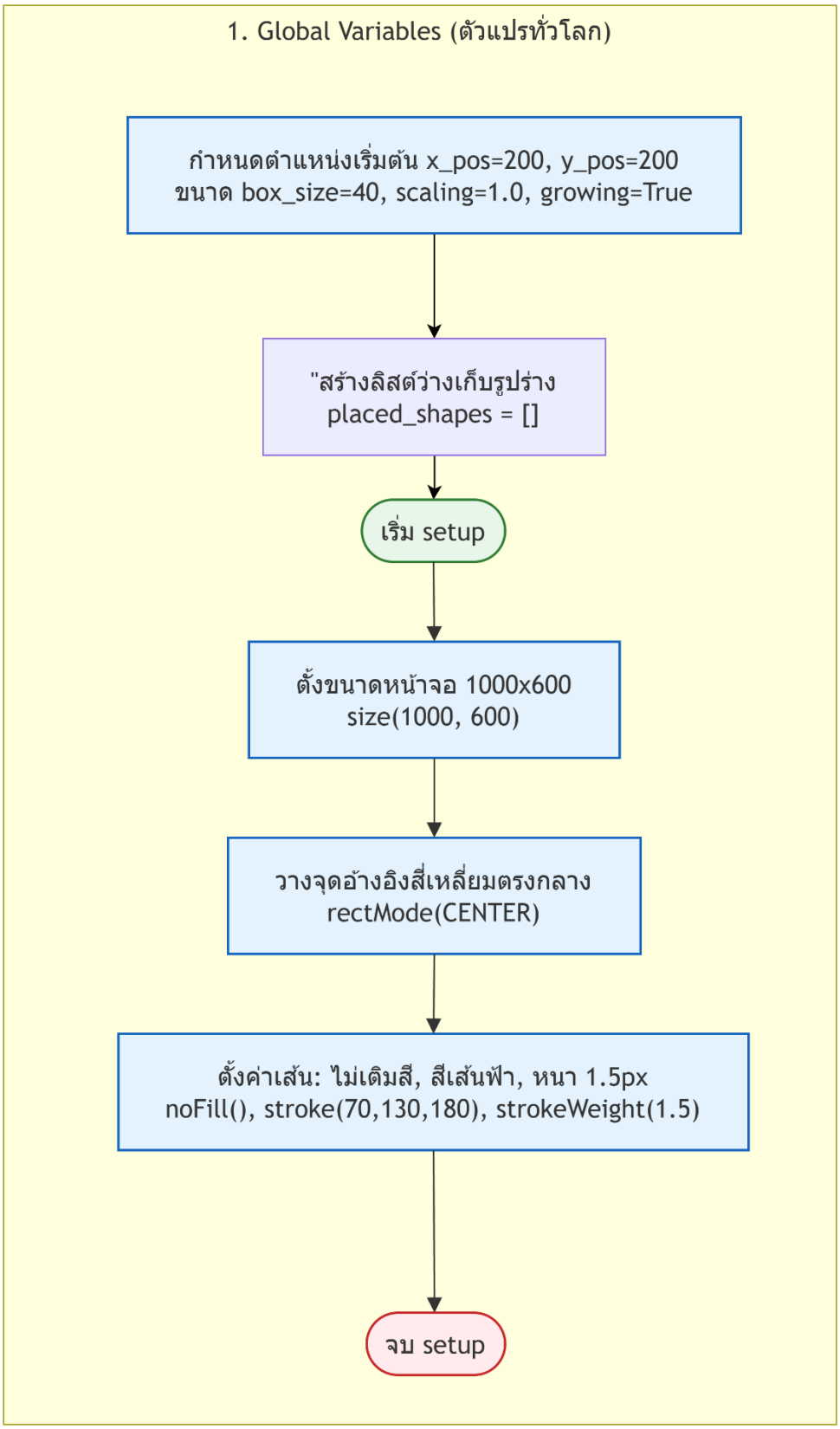
1. **การเคลียร์หน้าจอและการวาดรูปทรงสะสม:** ล้างผืนผ้าใบให้เป็นสีขาวในทุกเฟรม จากนั้นวนลูปดึงข้อมูลตำแหน่งและขนาดของเครื่องหมายบวกทั้งหมดที่เคยถูกบันทึกไว้ใน List ออกมาวาดซ้ำ เพื่อให้รูปที่ประทับไว้เดิมยังคงแสดงอยู่บนหน้าจอ
2. **เงื่อนไขทางเลือกแบบซ้อนกันในการย่อ-ขยาย**
    - ตรวจสอบสถานะว่ากำลังขยายตัวหรือไม่ หากกำลังขยายตัว ค่าสเกลจะเพิ่มขึ้นทีละนิดจนกระทั่งถึงจุดสูงสุดที่กำหนดไว้ (1.5 เท่า) ระบบจะทำการเปลี่ยนสถานะเป็นเท็จ เพื่อสั่งให้เริ่มย่อตัว

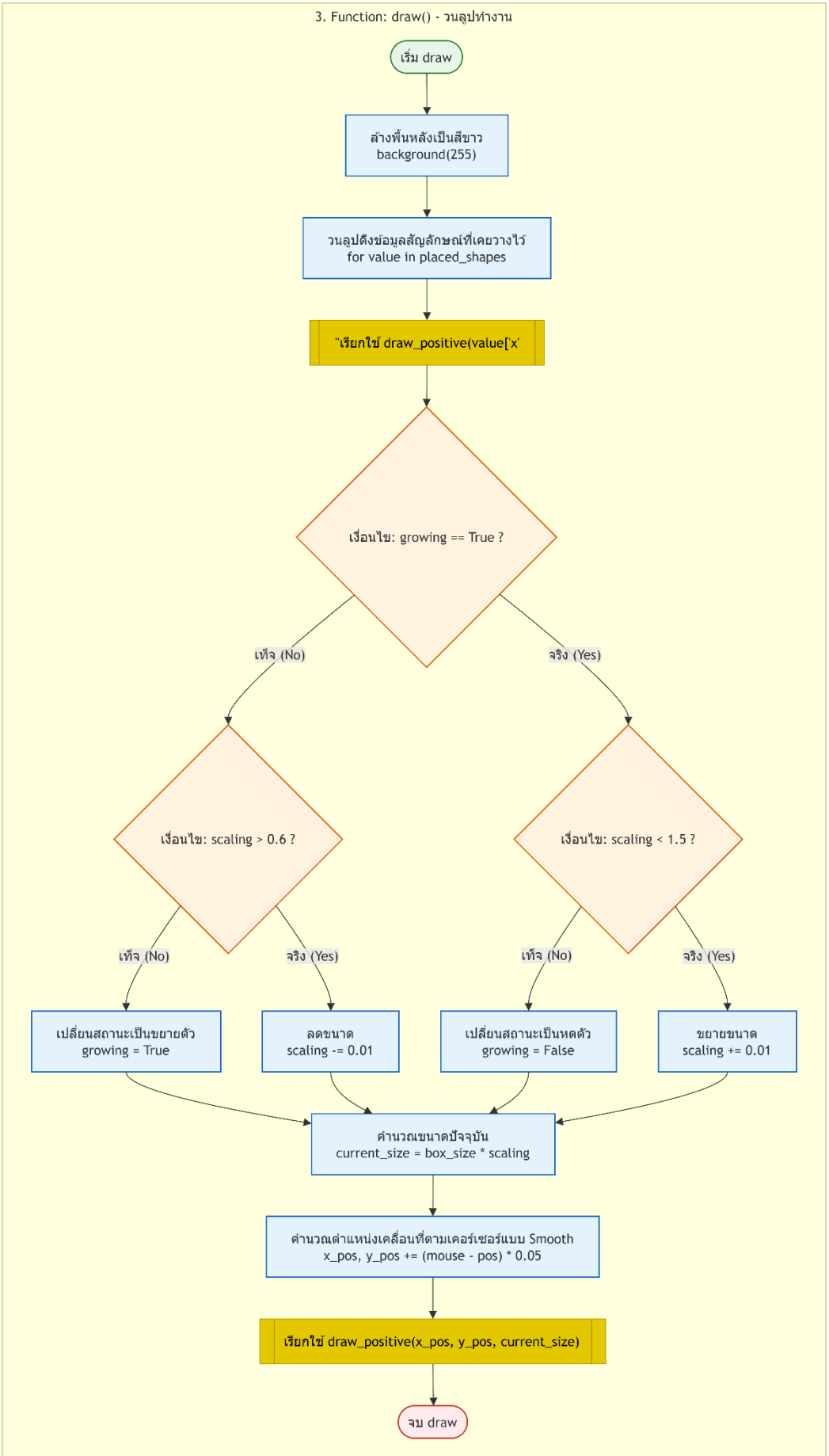
    - ในทางกลับกัน หากกำลังย่อตัว ค่าสเกลจะลดลงทีละนิด จนกระทั่งถึงจุดต่ำสุดที่กำหนดไว้ (0.6 เท่า) ระบบจะเปลี่ยนสถานะกลับเป็นจริง เพื่อสั่งให้เริ่มขยายตัวอีกครั้ง
    - นำค่าสเกลปัจจุบันไปคูณกับขนาดพื้นฐานเพื่อคำนวณเป็นขนาดจริงในเฟรมนั้นๆ
3. **ระบบการเคลื่อนที่ตามเมาส์:** คำนวณพิกัดใหม่โดยให้รูปทรงขยับเข้าหาตำแหน่งของเคอร์เซอร์เมาส์ทีละ 5% ของระยะห่างที่เหลืออยู่ ส่งผลให้การเคลื่อนที่ดูมีน้ำหนักและนุ่มนวล ไม่ติดเคอร์เซอร์แข็งจนเกินไป
4. **การแสดงผลรูปทรงหลัก:** วาดรูปทรงเครื่องหมายบวกที่มีขนาดและพิกัดล่าสุดลงบนหน้าจอตามตำแหน่งเคอร์เซอร์

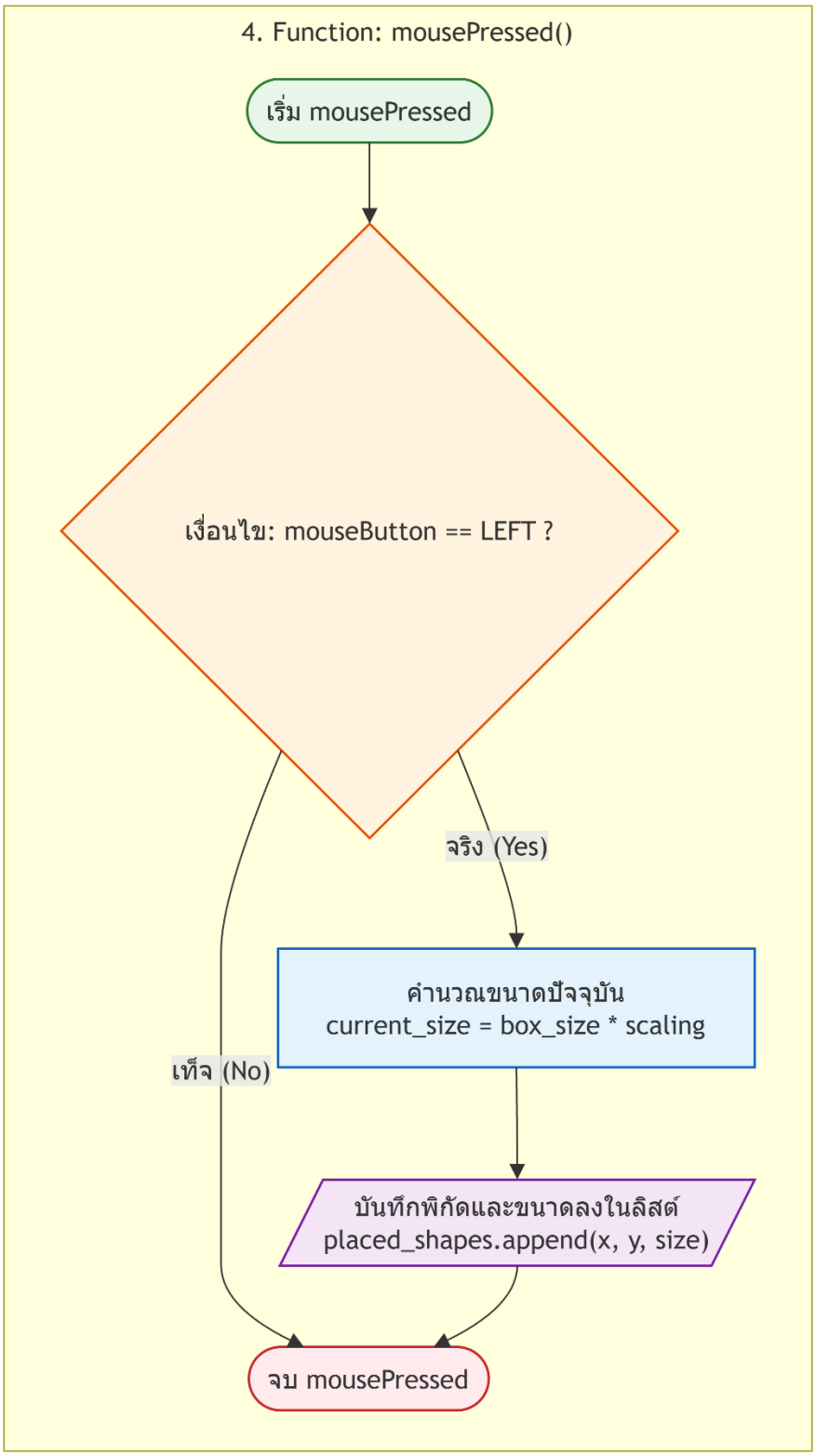
5. การโต้ตอบกับผู้ใช้ (User Interaction)

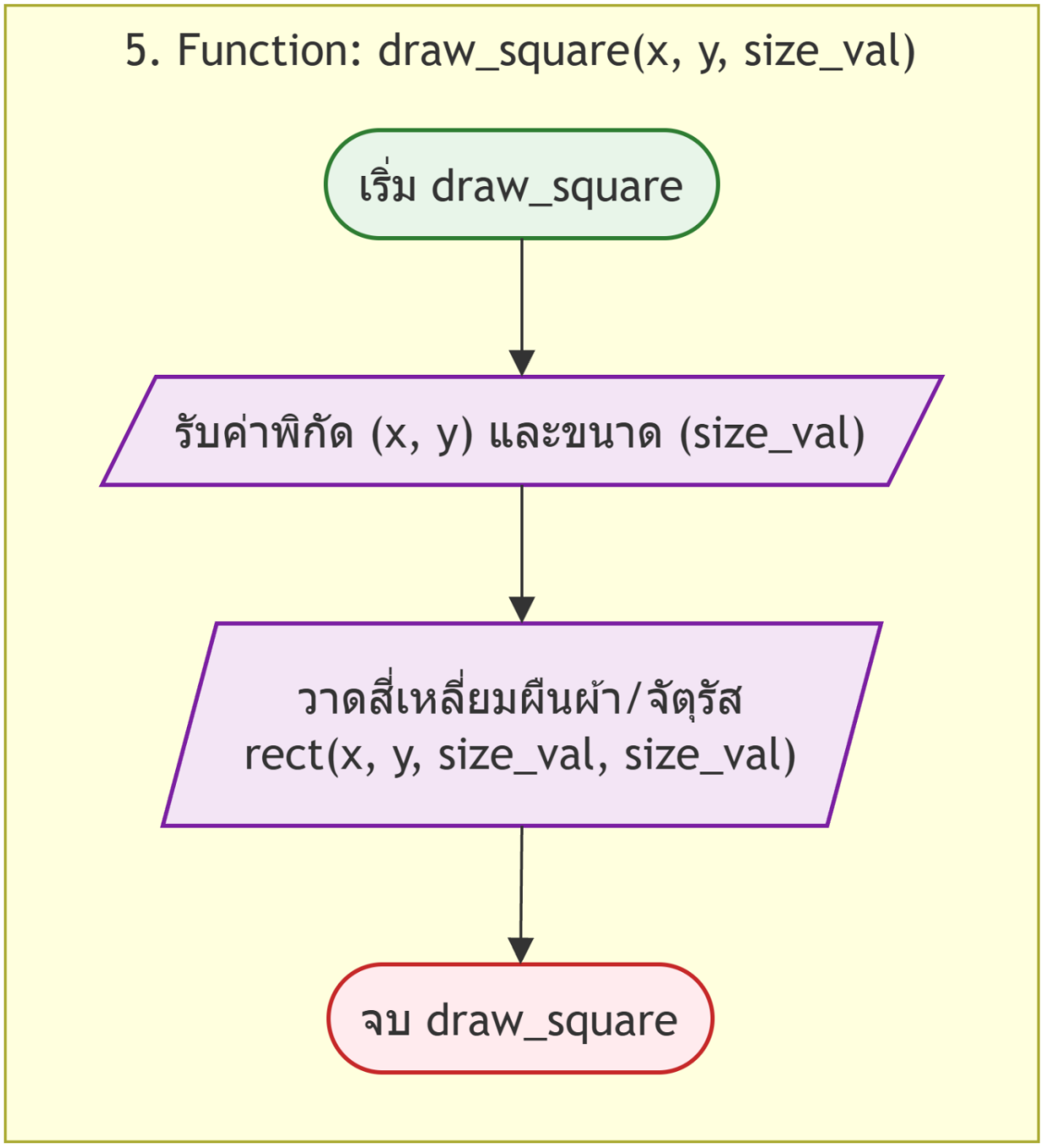
- **ฟังก์ชันรับการกดเมาส์:** เมื่อผู้ใช้ทำการกดปุ่มเมาส์ ระบบจะทำการเช็กว่าเป็น การคลิกเมาส์ปุ่มซ้ายหรือไม่
- **การบันทึกข้อมูล:** หากเป็นการคลิกซ้าย โปรแกรมจะดึงค่าพิกัดตำแหน่งล่าสุด และขนาดจริงที่ได้จากการคำนวณสเกลในขณะนั้น จัดเก็บเป็นโครงสร้างข้อมูลประเภทพจนานุกรม (Dictionary) แล้วนำไปต่อท้ายไว้ใน List รูปทรงที่ถูกวาง เพื่อให้ฟังก์ชันการวาดหลักนำไปแสดงผลค้างไว้บนหน้าจออย่างเป็นถาวร

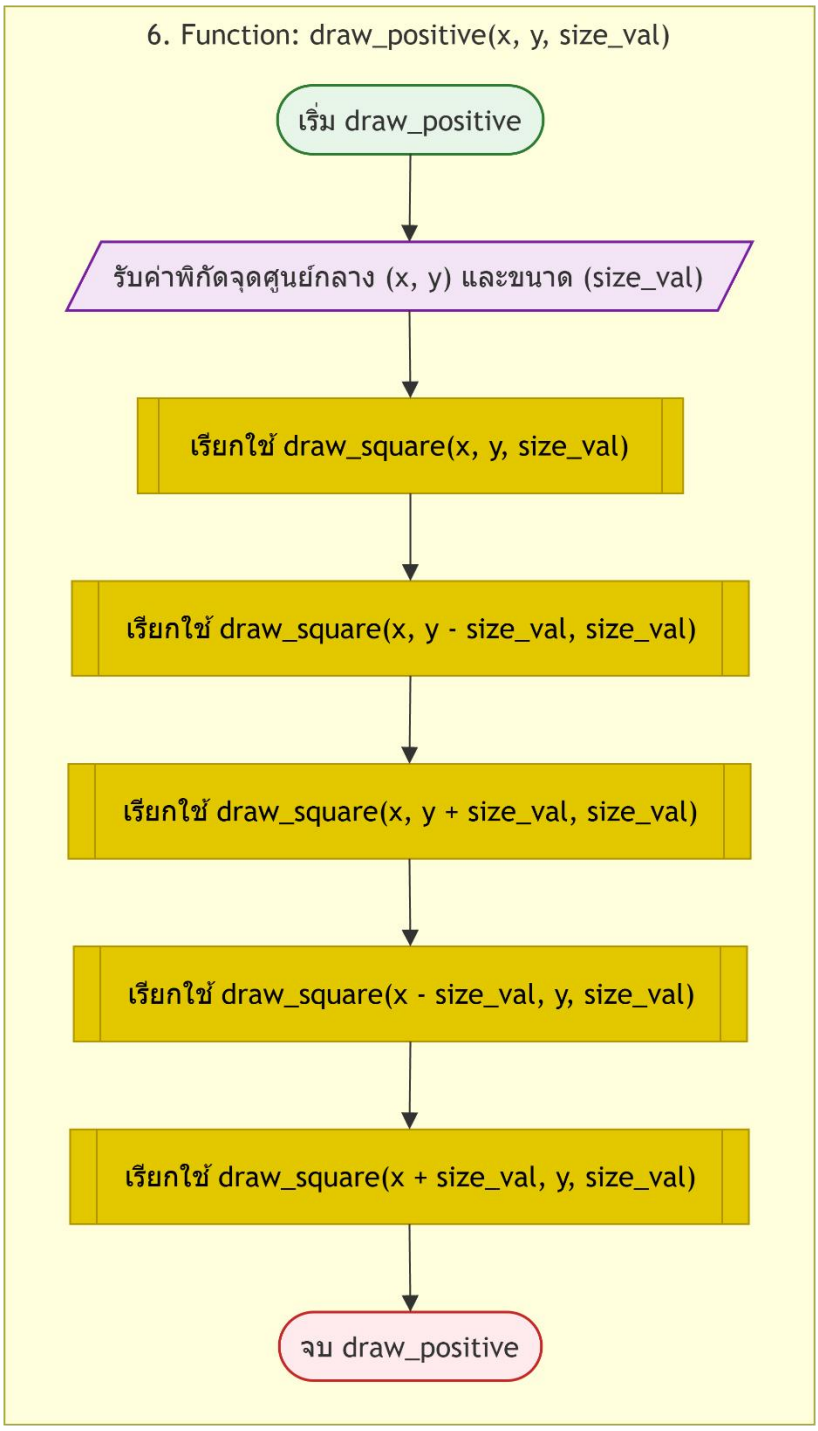
Flowchart


1. Global Variables (ตัวแปรทั่วโลก)
กำหนดตำแหน่งเริ่มต้น x_pos=200, y_pos=200
ขนาด box_size=40, scaling=1.0, growing=True
"สร้างลิสต์ว่างเก็บรูปร่าง
placed_shapes = []
เริ่ม setup
ตั้งขนาดหน้าจอ 1000x600
size(1000, 600)
วางจุดอ้างอิงสี่เหลี่ยมตรงกลาง
rectMode(CENTER)
ตั้งค่าเส้น: ไม่เติมสี, สีเส้นฟ้า, หนา 1.5px
noFill(), stroke(70,130,180), strokeWeight(1.5)
จบ setup


3. Function: draw() - วนลูปทำงาน
เริ่ม draw
ล้างพื้นหลังเป็นสีขาว
background(255)
วนลูปดึงข้อมูลสัญลักษณ์ที่เคยวางไว้
for value in placed_shapes
"เรียกใช้ draw_positive(value['x'
เงื่อนไข: growing == True ?
เท็จ (No)
จริง (Yes)
เงื่อนไข: scaling > 0.6 ?
เงื่อนไข: scaling < 1.5 ?
เท็จ (No)
จริง (Yes)
เท็จ (No)
จริง (Yes)
เปลี่ยนสถานะเป็นขยายตัว
growing = True
ลดขนาด
scaling -= 0.01
เปลี่ยนสถานะเป็นหดตัว
growing = False
ขยายขนาด
scaling += 0.01
คำนวณขนาดปัจจุบัน
current_size = box_size * scaling
คำนวณตำแหน่งเคลื่อนที่ตามเคอร์เซอร์แบบ Smooth
x_pos, y_pos += (mouse - pos) * 0.05
เรียกใช้ draw_positive(x_pos, y_pos, current_size)
จบ draw


4. Function: mousePressed()
เริ่ม mousePressed
เงื่อนไข: mouseButton == LEFT ?
จริง (Yes)
เท็จ (No)
คำนวณขนาดปัจจุบัน
current_size = box_size * scaling
บันทึกพิกัดและขนาดลงในลิสต์
placed_shapes.append(x, y, size)
จบ mousePressed


5. Function: draw_square(x, y, size_val)
เริ่ม draw_square
รับค่าพิกัด (x, y) และขนาด (size_val)
วาดสี่เหลี่ยมผืนผ้า/จัตุรัส
rect(x, y, size_val, size_val)
จบ draw_square


6. Function: draw_positive(x, y, size_val)
เริ่ม draw_positive
รับค่าพิกัดจุดศูนย์กลาง (x, y) และขนาด (size_val)
เรียกใช้ draw_square(x, y, size_val)
เรียกใช้ draw_square(x, y - size_val, size_val)
เรียกใช้ draw_square(x, y + size_val, size_val)
เรียกใช้ draw_square(x - size_val, y, size_val)
เรียกใช้ draw_square(x + size_val, y, size_val)
จบ draw_positive

Code

```
#Global variables for animation and interaction
x_pos = 200
y_pos = 200
box_size = 40
scaling = 1.0
growing = True

#Each element will be a dictionary containing: {'x': x, 'y': y, 'size': size}
placed_shapes = []

def setup():
    size(1000, 600)
    rectMode(CENTER)
    noFill()
    stroke(70, 130, 180)
    strokeWeight(1.5)
def draw():
    global x_pos, y_pos, box_size, scaling, growing
    background(255)

    #Draw permanently placed shapes
    # Loop through the list and redraw all previously placed plus signs
    for value in placed_shapes:
        draw_positive(value['x'], value['y'], value['size'])

    #Animation logic
    if growing:
        if scaling < 1.5:
            scaling += 0.01
        else:
            growing = False
    else:
        if scaling > 0.6:
            scaling -= 0.01
        else:
            growing = True

    current_size = box_size * scaling
    #Interaction logic : follow cursor
    x_pos += (mouseX - x_pos) * 0.05
    y_pos += (mouseY - y_pos) * 0.05

    #Draw the active floating plus sign that follows the cursor
    draw_positive(x_pos, y_pos, current_size)
```

```
def mousePressed():
    global x_pos, y_pos, box_size, scaling

    if mouseButton == LEFT:
        current_size = box_size * scaling

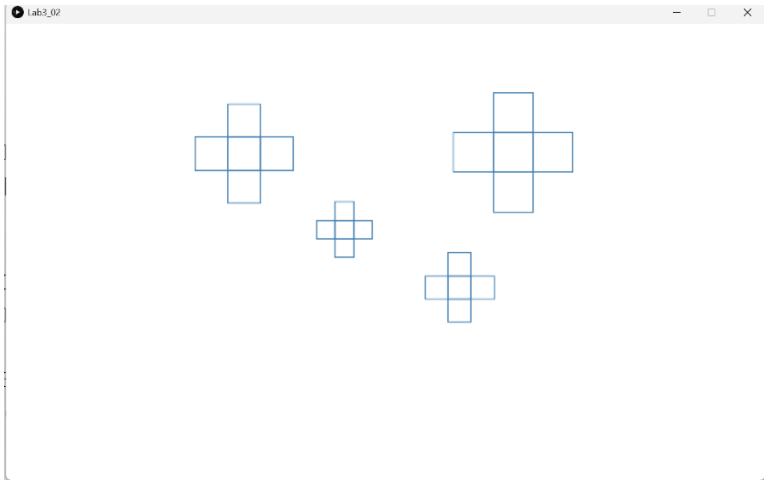
        # Save the current state (position and current size) into the list
        placed_shapes.append({
            'x': x_pos,
            'y': y_pos,
            'size': current_size
        })
```

```
#Function to draw a single square
def draw_square(x, y, size_val):
    rect(x, y, size_val, size_val)

#Function to draw a plus sign using draw_square()
def draw_positive(x, y, size_val):
    draw_square(x, y, size_val)
    draw_square(x, y - size_val, size_val)
    draw_square(x, y + size_val, size_val)
    draw_square(x - size_val, y, size_val)
    draw_square(x + size_val, y, size_val)
```

Lab3_02





ข้อที่ 3

โปรแกรมนี้สร้างภาพเคลื่อนไหวเชิงโต้ตอบเป็นรูป แมงกะพรุน ที่วาดขึ้นจากคำสั่งรูปทรงพื้นฐานอย่างวงรีและเส้นตรง ตัวแมงกะพรุนจะเคลื่อนที่ตามเคอร์เซอร์เมาส์ด้วยความนุ่มนวล โดยมีสายหนวดที่พริ้วไหวตามฟังก์ชันทางคณิตศาสตร์อย่างเป็นธรรมชาติ พร้อมระบบการโต้ตอบ 2 รูปแบบ คือ การปล่อยคลื่นช็อกเวฟเมื่อกดคลิกเมาส์ซ้าย และการเปิด-ปิดโหมดพรางตัวเมื่อกดคลิกเมาส์ขวา

2. การตั้งค่าเริ่มต้นและตัวแปรควบคุม

- **ตัวแปรระดับโกลบอล:**
    - **ตัวแปรตำแหน่งและชะลอการเคลื่อนที่:** เก็บพิกัดปัจจุบันของแมงกะพรุน และค่าหน่วงเวลาสำหรับการเคลื่อนที่ตามเมาส์
    - **ตัวแปรกำหนดจังหวะเวลา:** เก็บค่าเวลาที่เพิ่มขึ้นเรื่อยๆ สำหรับนำไปคำนวณการโบกสะบัดของหนวด
    - **ตัวแปรสถานะการโต้ตอบ:** เก็บค่ารัศมีของคลื่นช็อกเวฟ และค่าความจริง/เท็จ (Boolean) สำหรับสลับสถานะโหมดพรางตัว
- **ฟังก์ชันการตั้งค่าเริ่มต้น setup()**
    - กำหนดขนาดผืนผ้าใบเป็นรูปสี่เหลี่ยมจัตุรัสขนาด 600 x 600 พิกเซล
    - กำหนดความหนาของเส้นขอบเป็น 2 พิกเซล
    - ตั้งค่าความเร็วการแสดงผลไว้ที่ 60 เฟรมต่อวินาที เพื่อให้ภาพเคลื่อนไหวมีความลื่นไหล

3. การคำนวณใน draw()

ในฟังก์ชันการทำงานหลักที่ประมวลผลวนลูปอย่างต่อเนื่อง มีลำดับขั้นตอนดังนี้:

1. **การลงสีพื้นหลัง:** ระบายสีพื้นหลังเป็นโทนสีน้ำเงินเข้มในทุกเฟรม เพื่อจำลองบรรยากาศท้องทะเลลึกหรืออวกาศ
2. **ระบบการเคลื่อนที่ตามเมาส์แบบนุ่มนวล (Easing Movement):** คำนวณพิกัดใหม่โดยนำระยะห่างระหว่างตำแหน่งปัจจุบันกับตำแหน่งเมาส์มาคูณกับค่าหน่วง ทำให้แมงกะพรุนขยับตามเมาส์อย่างมีความนุ่มนวลและไม่ติดเคอร์เซอร์แข็งจนเกินไป
3. **การอัปเดตตัวนับเวลา:** เพิ่มค่าตัวแปรเวลาอย่างต่อเนื่องทีละน้อย เพื่อใช้อ้างอิงการพริ้วไหวของหนวดแมงกะพรุน

4. กลไกการสร้างเอฟเฟกต์คลื่นช็อกเวฟ

- **การตรวจสอบเงื่อนไข:** โปรแกรมจะเช็กว่าค่ารัศมีของคลื่นช็อกเวฟมากกว่าศูนย์หรือไม่ หากมากกว่าศูนย์จะทำการวาดวงรีขยายวงออกจากจุดศูนย์กลางของแมงกะพรุน
- **การจางหายอย่างสมบูรณ์:** ความโปร่งแสงของเส้นขอบวงรีจะลดลงตามขนาดรัศมีที่ขยายใหญ่ขึ้น ทำให้คลื่นค่อยๆ จางหายไปอย่างราบรื่น
- **การรีเซ็ตวงรอบ:** เมื่อรัศมีขยายตัวจนเกินขนาดขีดจำกัดที่กำหนดไว้ (120 พิกเซล) ค่ารัศมีจะถูกตั้งกลับเป็นศูนย์เพื่อหยุดการวาดคลื่น

5. ระบบการโต้ตอบผ่านเมาส์

โปรแกรมใช้ฟังก์ชันรับเหตุการณ์การกดเมาส์ เพื่อจำแนกการทำงานตามปุ่มที่ถูกกด:

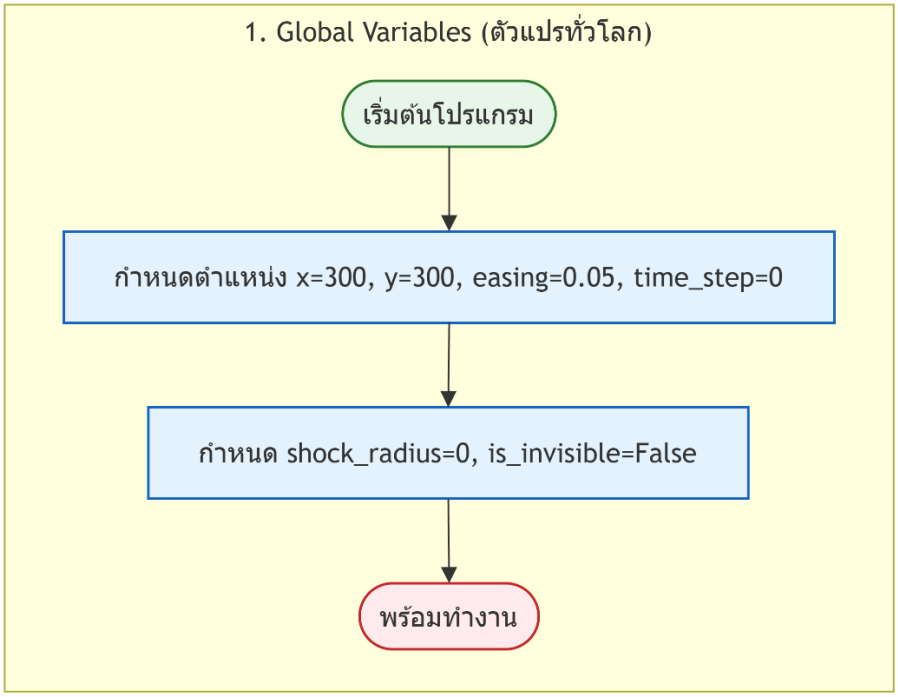
- **เมื่อกดปุ่มเมาส์ซ้าย** กำหนดค่ารัศมีเริ่มต้นของคลื่นช็อกเวฟ เพื่อปล่อยวงคลื่นพลังงานกระจายออกจากตัวแมงกะพรุน
- **เมื่อกดปุ่มเมาส์ขวา** ทำการสลับสถานะของตัวแปรโหมดพรางตัว ให้เปลี่ยนสลับระหว่างเปิดและปิด

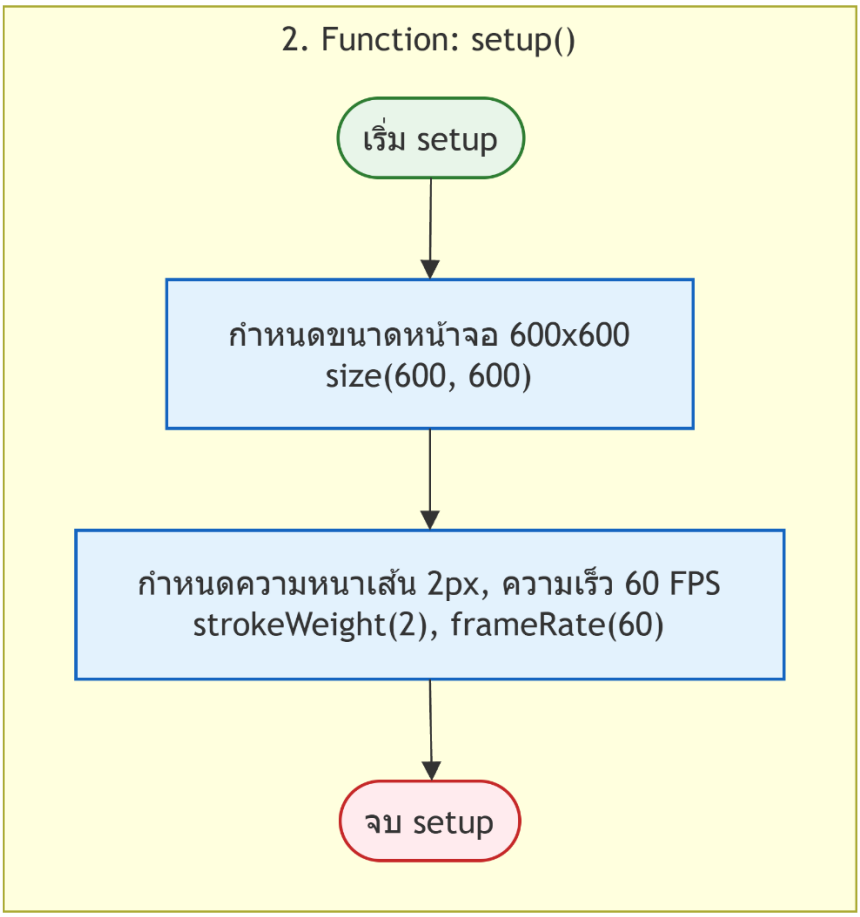
6. โครงสร้างฟังก์ชันย่อยในการวาดรูปทรง

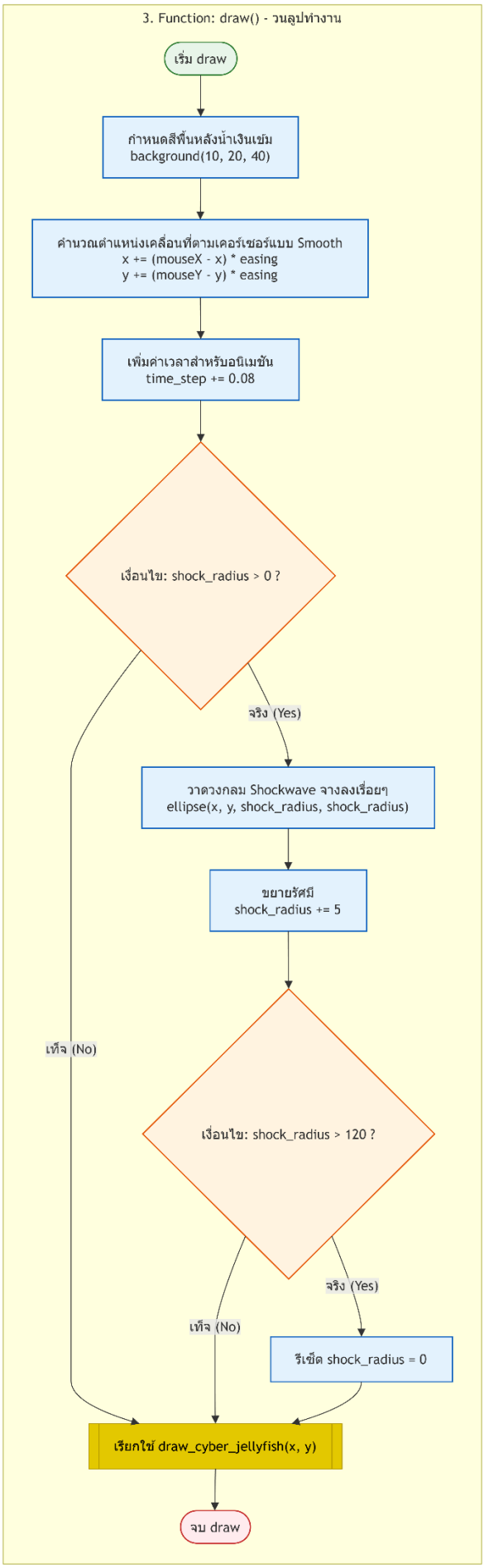
โปรแกรมถูกออกแบบตามหลัก Modular Design โดยแบ่งฟังก์ชันย่อยออกเป็น 3 ส่วนหลัก เพื่อสร้างความมีระเบียบในการทำงาน:

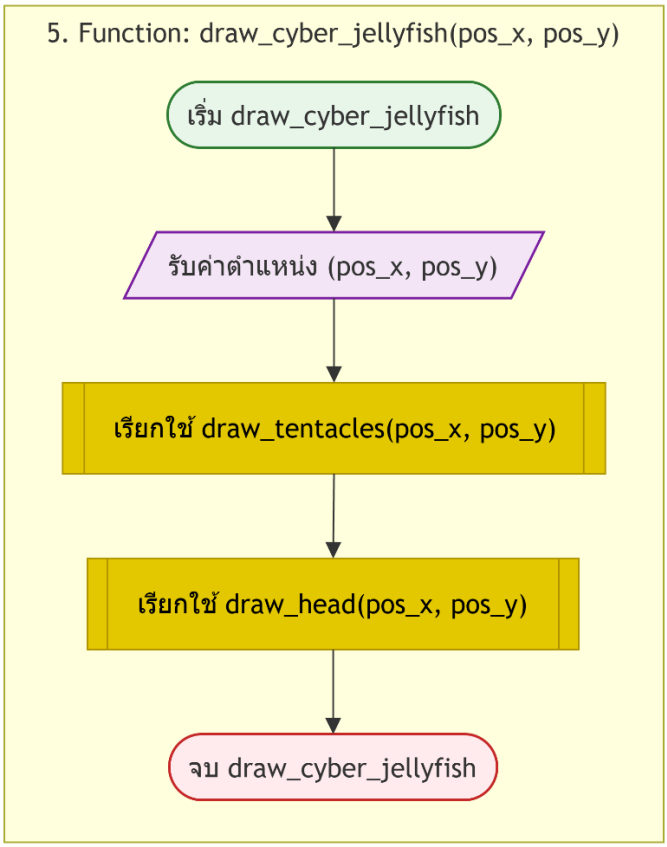
1. **ฟังก์ชันควบคุมการวาดรวม** ทำหน้าที่เป็นตัวกลางในการเรียกลำดับการวาด โดยสั่งให้วาดส่วนหนวดก่อน แล้วจึงวาดส่วนหัวทับด้านบน เพื่อให้ชั้นของภาพ ซ้อนทับกันอย่างถูกต้อง
2. **ฟังก์ชันวาดส่วนหัว**
    - **เมื่ออยู่ในโหมดพรางตัว:** วาดเฉพาะเส้นขอบของวงรีด้วยโทนสีฟ้าสว่างที่มีความโปร่งแสงสูง และปิดการใส่สีพื้นหลัง เพื่อให้ตัวแมงกะพรุนดูโปร่งใส
    - **เมื่ออยู่ในโหมดปกติ:** วาดวงรีซ้อนกันหลายชั้น โดยระบายสีพื้นหลังทึบแสงบางส่วน ใช้โทนสีฟ้าและสีชมพูสว่างตรงแกนกลาง พร้อมวาดวงรีสีขาวขนาดเล็กเป็นจุดตา
3. **ฟังก์ชันวาดส่วนหนวด**
    - ใช้วงลูปทำงานซ้ำเพื่อสร้างเส้นหนวดทั้งหมด 5 เส้น ซึ่งจัดวางกระจายตามแนวราบด้านใต้ส่วนหัว
    - ใช้ฟังก์ชันทางคณิตศาสตร์ ไซน์ (Sine) โดยนำค่าเวลาและดรรชนีของหนวดแต่ละเส้นมาคำนวณ เพื่อสร้างระยะเหวี่ยงกวาดไปมา ทำให้หนวดแต่ละเส้นสะบัดพริ้วไหวด้วยจังหวะที่เหลื่อมกันอย่างเป็นธรรมชาติ
    - ใช้คำสั่งวาดเส้นตรงต่อจากส่วนหัวลงมา และวาดวงรีขนาดเล็กไว้ที่ปลายหนวดแต่ละเส้น โดยค่าความโปร่งแสงและการใส่สีจะปรับเปลี่ยนไปตามสถานะของโหมดพรางตัวเช่นกัน

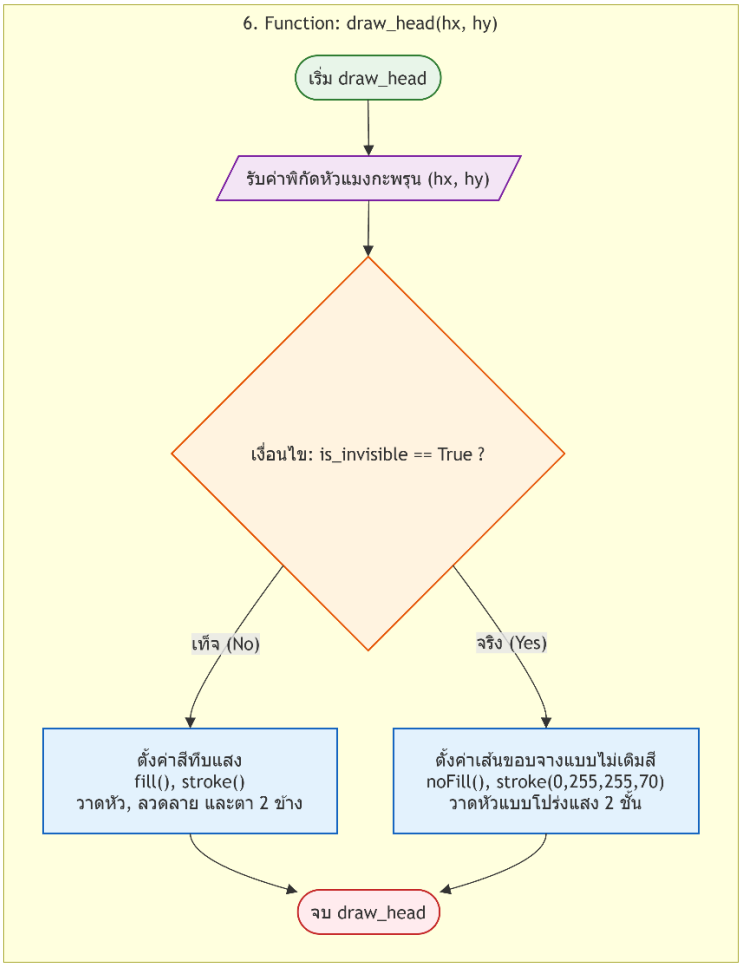
Flowchart

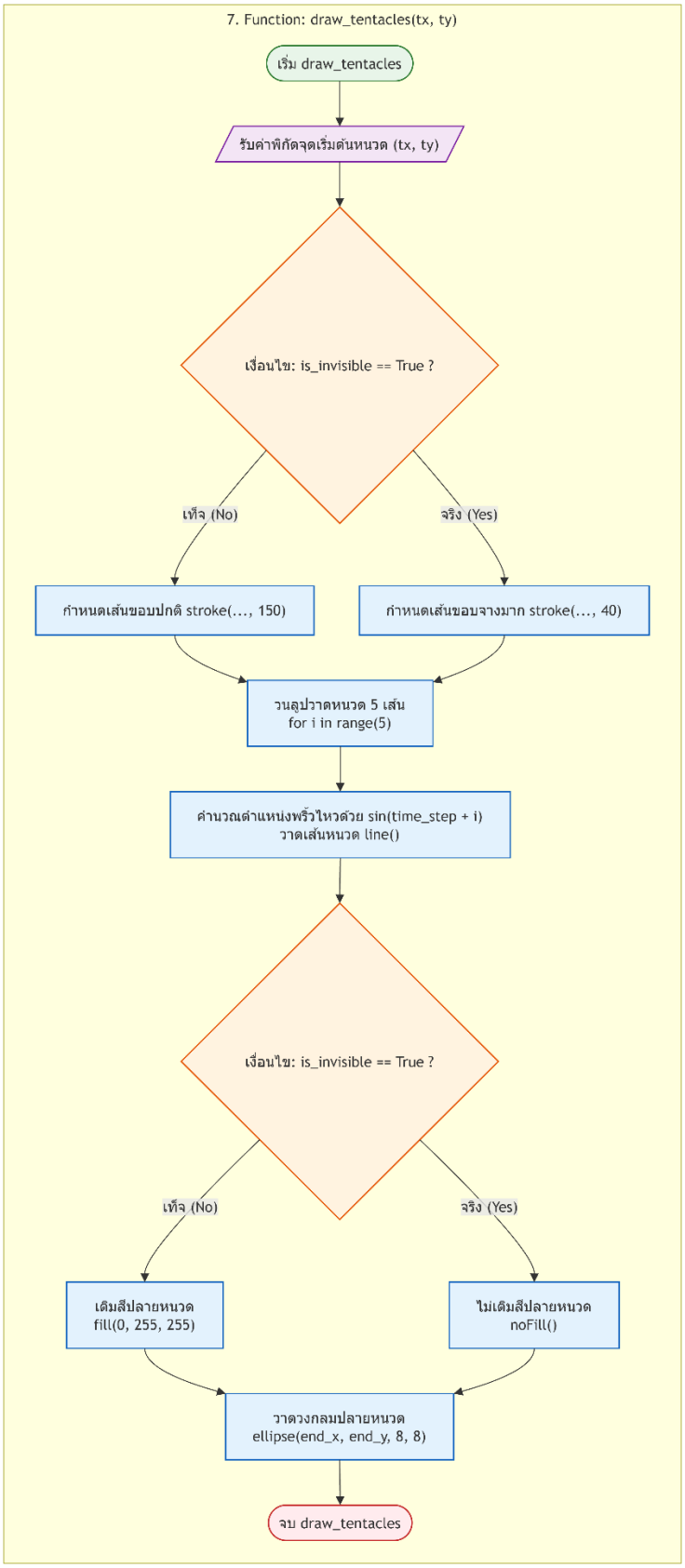

1. Global Variables (ตัวแปรทั่วโลก)
เริ่มต้นโปรแกรม
กำหนดตำแหน่ง x=300, y=300, easing=0.05, time_step=0
กำหนด shock_radius=0, is_invisible=False
พร้อมทำงาน


2. Function: setup()
เริ่ม setup
กำหนดขนาดหน้าจอ 600x600
size(600, 600)
กำหนดความหนาเส้น 2px, ความเร็ว 60 FPS
strokeWeight(2), frameRate(60)
จบ setup


3. Function: draw() - วนลูปทำงาน
เริ่ม draw
กำหนดสีพื้นหลังน้ำเงินเข้ม
background(10, 20, 40)
คำนวณตำแหน่งเคลื่อนที่ตามเคอร์เซอร์แบบ Smooth
x += (mouseX - x) * easing
y += (mouseY - y) * easing
เพิ่มค่าเวลาสำหรับอนิเมชัน
time_step += 0.08
เงื่อนไข: shock_radius > 0 ?
จริง (Yes)
เท็จ (No)
วาดวงกลม Shockwave จางลงเรื่อยๆ
ellipse(x, y, shock_radius, shock_radius)
ขยายรัศมี
shock_radius += 5
เงื่อนไข: shock_radius > 120 ?
จริง (Yes)
เท็จ (No)
รีเซ็ต shock_radius = 0
เรียกใช้ draw_cyber_jellyfish(x, y)
จบ draw


5. Function: draw_cyber_jellyfish(pos_x, pos_y)
เริ่ม draw_cyber_jellyfish
รับค่าตำแหน่ง (pos_x, pos_y)
เรียกใช้ draw_tentacles(pos_x, pos_y)
เรียกใช้ draw_head(pos_x, pos_y)
จบ draw_cyber_jellyfish


6. Function: draw_head(hx, hy)
เริ่ม draw_head
รับค่าพิกัดหัวแมงกะพรุน (hx, hy)
เงื่อนไข: is_invisible == True ?
เท็จ (No)
จริง (Yes)
ตั้งค่าสีทึบแสง
fill(), stroke()
วาดหัว, ลวดลาย และตา 2 ข้าง
ตั้งค่าเส้นขอบจางแบบไม่เติมสี
noFill(), stroke(0,255,255,70)
วาดหัวแบบโปร่งแสง 2 ชั้น
จบ draw_head


7. Function: draw_tentacles(tx, ty)
เริ่ม draw_tentacles
รับค่าพิกัดจุดเริ่มต้นหนวด (tx, ty)
เงื่อนไข: is_invisible == True ?
เท็จ (No)
จริง (Yes)
กำหนดเส้นขอบปกติ stroke(..., 150)
กำหนดเส้นขอบจางมาก stroke(..., 40)
วนลูปวาดหนวด 5 เส้น
for i in range(5)
คำนวณตำแหน่งพริ้วไหวด้วย sin(time_step + i)
วาดเส้นหนวด line()
เงื่อนไข: is_invisible == True ?
เท็จ (No)
จริง (Yes)
เติมสีปลายหนวด
fill(0, 255, 255)
ไม่เติมสีปลายหนวด
noFill()
วาดวงกลมปลายหนวด
ellipse(end_x, end_y, 8, 8)
จบ draw_tentacles

Code

```
x = 300
y = 300
easing = 0.05
time_step = 0

#Interaction variables
shock_radius = 0      #For left click : shock wave
is_invisible = False  #For right click : invisible

def setup():
    size(600, 600)
    strokeWeight(2)
    frameRate(60)

def draw():
    global x, y, time_step, shock_radius

    background(10, 20, 40)

    #Follow cursor
    posX = mouseX
    posY = mouseY
    x += (posX - x) * easing
    y += (posY - y) * easing

    time_step += 0.08

    #Left click : shock wave
    #increase shock wave radius
    if shock_radius > 0:
        noFill()
        stroke(0, 255, 255, 255 - (shock_radius * 2)) #slowly disappear in linear
        ellipse(x, y, shock_radius, shock_radius)
        shock_radius += 5
        if shock_radius > 120: #max radius condition
            shock_radius = 0

    #draw jellyfish
    draw_cyber_jellyfish(x, y)

#Mouse interaction
def mousePressed():
    global shock_radius, is_invisible

    if mouseButton == LEFT:
        #shock wave
        shock_radius = 10

    elif mouseButton == RIGHT:
        #invisible mode
        is_invisible = not is_invisible

#Subroutines

def draw_cyber_jellyfish(pos_x, pos_y):
    draw_tentacles(pos_x, pos_y)
    draw_head(pos_x, pos_y)
```

```python
def draw_head(hx, hy):
    global is_invisible

    #Check for invisible mode
    if is_invisible:
        #if yes, fill inside with tranparent, left only stroke
        noFill()
        stroke(0, 255, 255, 70)
        ellipse(hx, hy, 100, 80)
        ellipse(hx, hy, 50, 40)
    else:
        #if no, fill normally
        fill(0, 255, 255, 100)
        stroke(0, 255, 255)
        ellipse(hx, hy, 100, 80)

        fill(255, 0, 128, 200)
        ellipse(hx, hy, 50, 40)
        fill(255)
        ellipse(hx - 15, hy, 10, 10)
        ellipse(hx + 15, hy, 10, 10)
```

```python
def draw_tentacles(tx, ty):
    global is_invisible

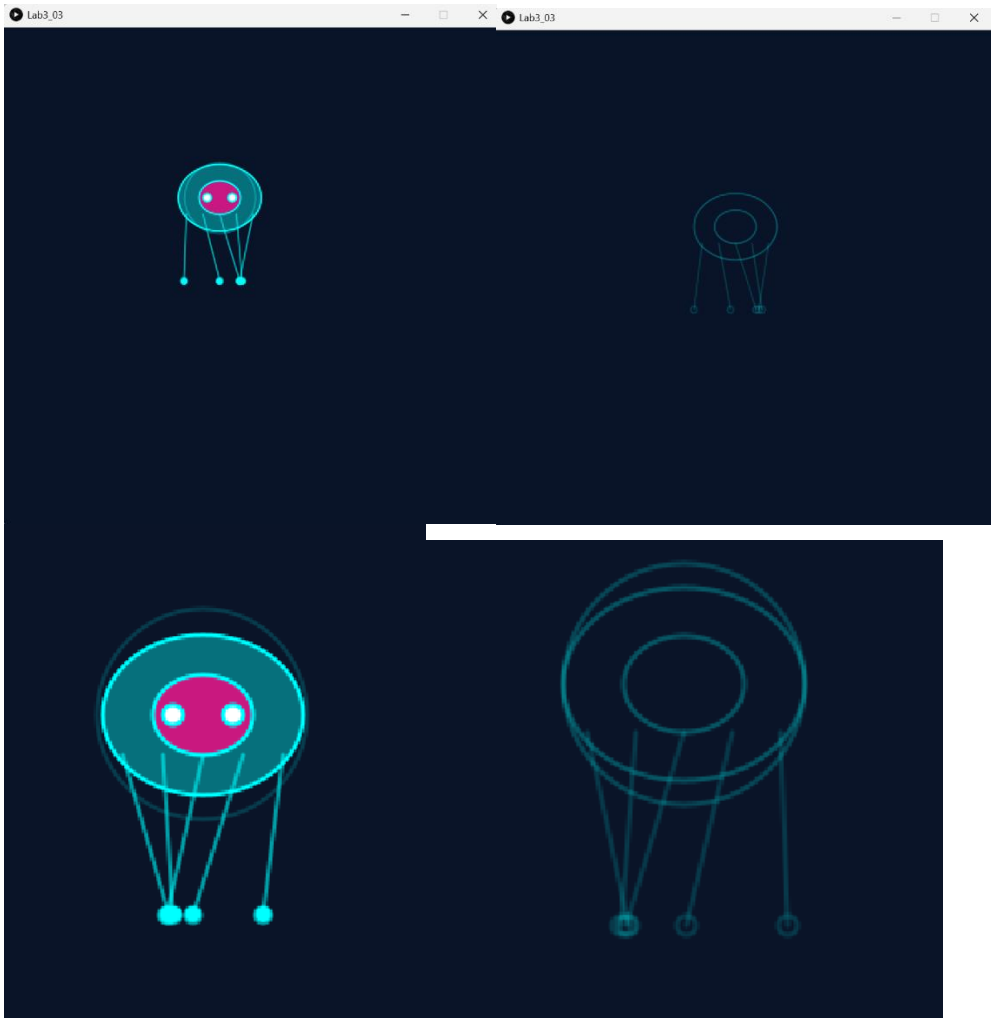
    #Check for invisible
    if is_invisible:
        stroke(0, 255, 255, 40)
    else:
        stroke(0, 255, 255, 150)

    for i in range(5):
        start_x = tx - 40 + (i * 20)
        start_y = ty + 20

        offset = sin(time_step + i) * 25
        end_x = start_x + offset
        end_y = start_y + 80

        line(start_x, start_y, end_x, end_y)

        if is_invisible:
            noFill()
        else:
            fill(0, 255, 255)
        ellipse(end_x, end_y, 8, 8)
```


Lab3_03
Lab3_03

Text Mode

ข้อที่ 1

โปรแกรมนี้ทำหน้าที่คำนวณหาค่าดัชนีมวลกาย (Body Mass Index: BMI) จากค่าน้ำหนักและส่วนสูงที่ผู้ใช้ป้อนเข้ามา จากนั้นจะนำค่าดัชนีมวลกายที่คำนวณได้มาวิเคราะห์และประเมินสภาวะทางร่างกายของผู้ใช้ตามเกณฑ์มาตรฐานสุขภาพ โดยแสดงผลลัพธ์เป็นค่าดัชนีมวลกายทศนิยมสองตำแหน่งพร้อมคำแปลผลสภาวะร่างกาย

1. ฟังก์ชันคำนวณดัชนีมวลกาย

- **การรับค่าเข้า:** ฟังก์ชันส่วนนี้รับข้อมูลนำเข้า 2 ค่า ได้แก่ น้ำหนักตัว (หน่วยเป็นกิโลกรัม) และส่วนสูง (หน่วยเป็นเซนติเมตร)
- **การแปลงหน่วยและคำนวณ:**
    - ทำการแปลงหน่วยส่วนสูงจากเซนติเมตรให้เป็นเมตร โดยนำส่วนสูงไปหารด้วย 100
    - นำค่าน้ำหนักตัวมาหารด้วยผลคูณของส่วนสูงในหน่วยเมตรยกกำลังสอง ตามสูตรมาตรฐานการคำนวณดัชนีมวลกาย
- **การส่งคืนค่า:** ส่งคืนผลลัพธ์ที่เป็นตัวเลขดัชนีมวลกายกลับไปให้จุดที่เรียกใช้งาน

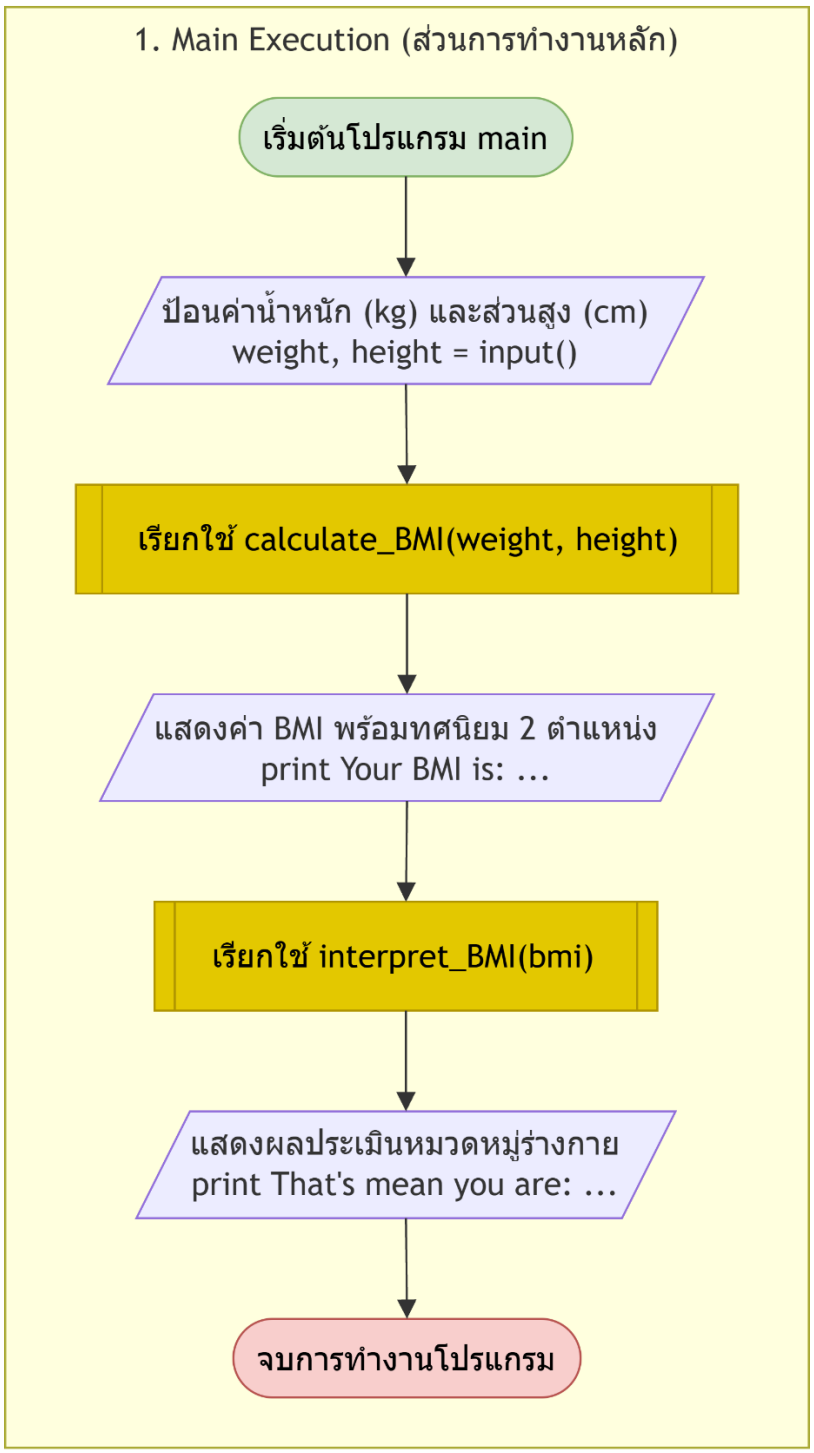
2. ฟังก์ชันแปลผลเกณฑ์สภาวะร่างกาย

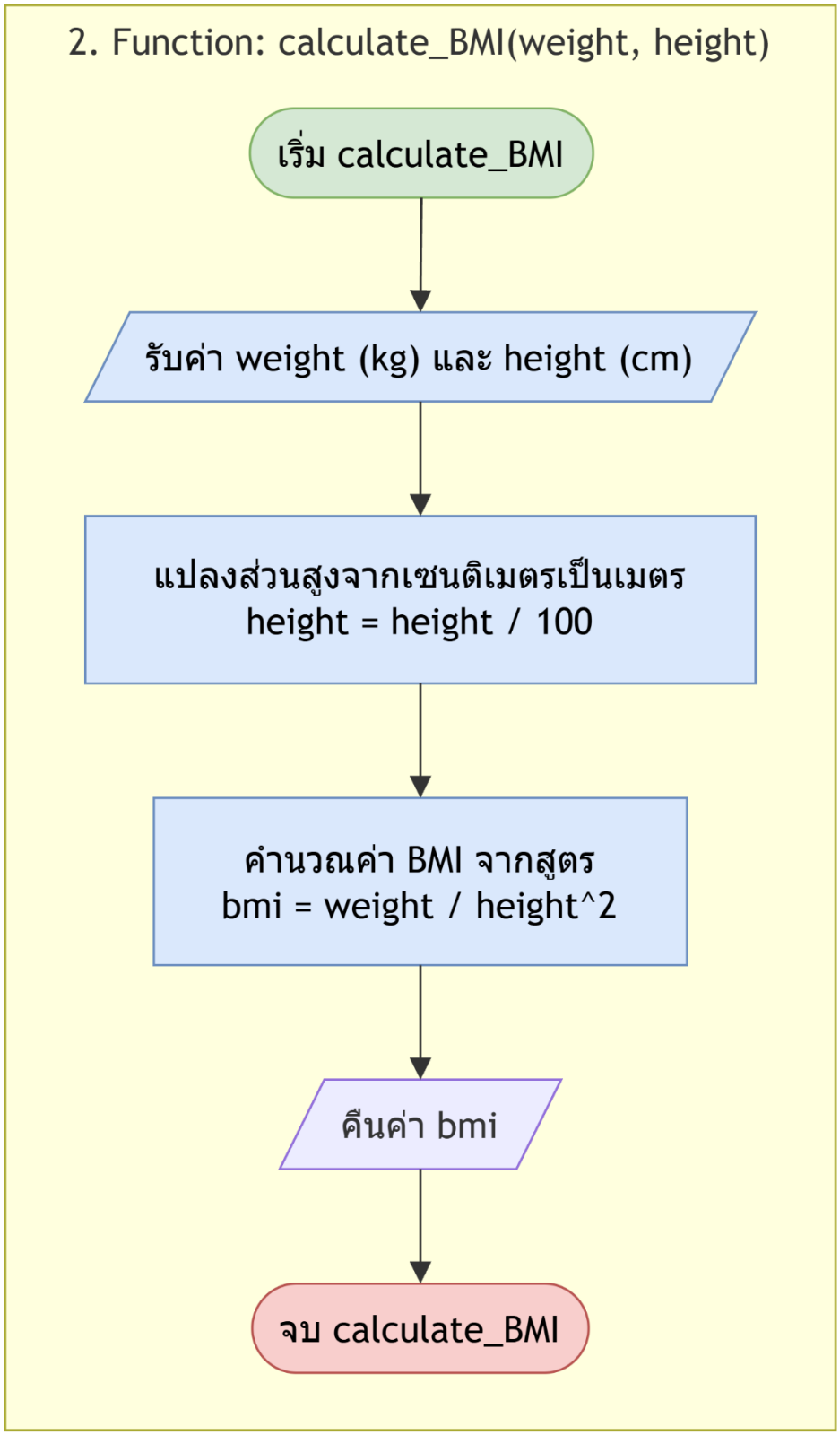
ฟังก์ชันส่วนนี้ทำหน้าที่รับค่าดัชนีมวลกายที่ได้จากการคำนวณ มาทำการตรวจสอบและเปรียบเทียบตามโครงสร้างเงื่อนไขทางเลือกแบบหลายทาง เพื่อจัดกลุ่มสภาวะร่างกายออกเป็น 6 ระดับ ดังนี้:

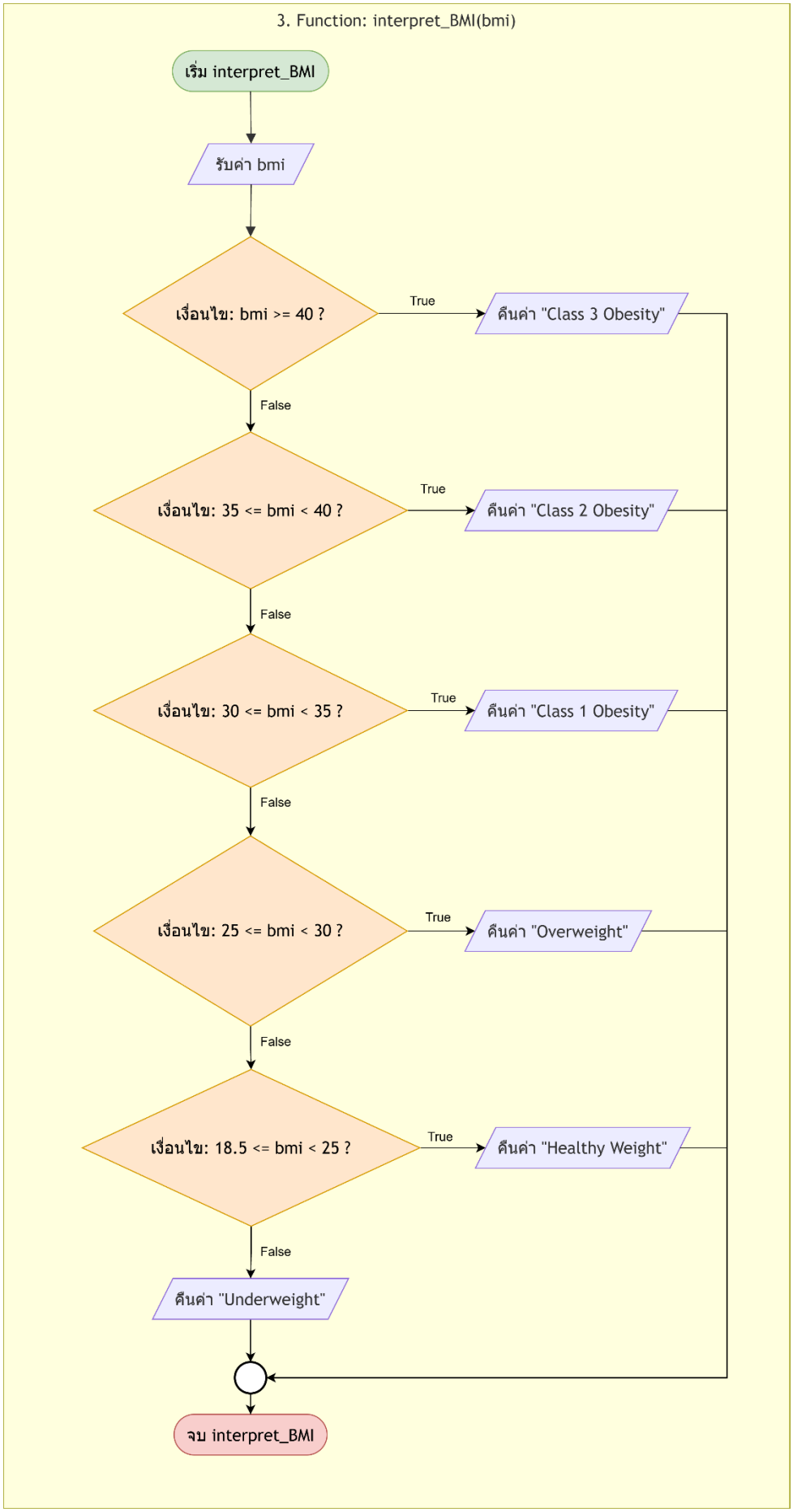
- **โรคอ้วนระดับ 3 (อ้วนรุนแรง):** สำหรับผู้ที่มีค่าดัชนีมวลกายตั้งแต่ 40 ขึ้นไป
- **โรคอ้วนระดับ 2:** สำหรับผู้ที่มีค่าดัชนีมวลกายอยู่ในช่วง 35 ถึงน้อยกว่า 40
- **โรคอ้วนระดับ 1:** สำหรับผู้ที่มีค่าดัชนีมวลกายอยู่ในช่วง 30 ถึงน้อยกว่า 35
- **น้ำหนักเกิน:** สำหรับผู้ที่มีค่าดัชนีมวลกายอยู่ในช่วง 25 ถึงน้อยกว่า 30
- **น้ำหนักเกณฑ์ปกติ (สุขภาพดี):** สำหรับผู้ที่มีค่าดัชนีมวลกายอยู่ในช่วง 18.5 ถึงน้อยกว่า 25
- **น้ำหนักน้อยกว่าเกณฑ์:** สำหรับผู้ที่มีค่าดัชนีมวลกายต่ำกว่า 18.5

เมื่อตรงกับเงื่อนไขใด ฟังก์ชันจะส่งคืนข้อความอธิบายสภาวะร่างกายในระดับนั้นๆ ออกมา

Flowchart


1. Main Execution (ส่วนการทำงานหลัก)
เริ่มต้นโปรแกรม main
ป้อนค่าน้ำหนัก (kg) และส่วนสูง (cm)
weight, height = input()
เรียกใช้ calculate_BMI(weight, height)
แสดงค่า BMI พร้อมทศนิยม 2 ตำแหน่ง
print Your BMI is: ...
เรียกใช้ interpret_BMI(bmi)
แสดงผลประเมินหมวดหมู่ร่างกาย
print That's mean you are: ...
จบการทำงานโปรแกรม


2. Function: calculate_BMI(weight, height)
เริ่ม calculate_BMI
รับค่า weight (kg) และ height (cm)
แปลงส่วนสูงจากเซนติเมตรเป็นเมตร
height = height / 100
คำนวณค่า BMI จากสูตร
bmi = weight / height^2
คืนค่า bmi
จบ calculate_BMI


3. Function: interpret_BMI(bmi)
เริ่ม interpret_BMI
รับค่า bmi
เงื่อนไข: bmi >= 40 ?
True
คืนค่า "Class 3 Obesity"
False
เงื่อนไข: 35 <= bmi < 40 ?
True
คืนค่า "Class 2 Obesity"
False
เงื่อนไข: 30 <= bmi < 35 ?
True
คืนค่า "Class 1 Obesity"
False
เงื่อนไข: 25 <= bmi < 30 ?
True
คืนค่า "Overweight"
False
เงื่อนไข: 18.5 <= bmi < 25 ?
True
คืนค่า "Healthy Weight"
False
คืนค่า "Underweight"
จบ interpret_BMI

Code

```python
def calculate_BMI(weight, height):
    height = height / 100
    bmi = weight / height**2
    return bmi

def interpret_BMI(bmi):
    if bmi >= 40:
        return "Class 3 Obesity (Severe Obesity)"
    elif bmi >= 35 and bmi < 40:
        return "Class 2 Obesity"
    elif bmi >= 30 and bmi < 35:
        return "Class 1 Obesity"
    elif bmi >= 25 and bmi < 30:
        return "Overweight"
    elif bmi >= 18.5 and bmi < 25:
        return "Healthy Weight"
    else:
        return "Underweight"

if __name__ == '__main__':
    weight, height = map(int, input("Enter your weight in kilogram\nand your height in centimeter: ").split())
    print(f"Your BMI is: {calculate_BMI(weight, height):.2f} kg/m^2")
    print(f"That's mean you are: {interpret_BMI(calculate_BMI(weight, height))}")
```

```
Enter your weight in kilogram
and your height in centimeter: 96 181
Your BMI is: 29.30 kg/m^2
That's mean you are: Overweight
Enter your weight in kilogram
and your height in centimeter: 45 170
Your BMI is: 15.57 kg/m^2
That's mean you are: Underweight
Enter your weight in kilogram
and your height in centimeter: 150 176
Your BMI is: 48.42 kg/m^2
That's mean you are: Class 3 Obesity (Severe Obesity)
```

ข้อที่ 2

1. ภาพรวมของโปรแกรม

โปรแกรมนี้ทำหน้าที่รับค่าคะแนนสอบจากผู้ใช้ ตรวจสอบความถูกต้องของขอบเขตข้อมูลให้อยู่ในช่วง 0 ถึง 100 จากนั้นนำคะแนนดังกล่าวไปทำการประเมินผลเพื่อตัดเกรดออกมาเป็นระดับอักขระตัวอักษร (A, B, C, D, F) และแสดงผลลัพธ์ออกทางหน้าจอ

2. ฟังก์ชันตรวจสอบเกณฑ์การตัดเกรด

ฟังก์ชันส่วนนี้รับค่าคะแนนเข้าประมวลผล และใช้นำมาเปรียบเทียบตามโครงสร้างเงื่อนไขทางเลือก (Conditional Statement) เพื่อส่งคืนผลลัพธ์เป็นระดับเกรดต่างๆ ดังนี้:

- **เกรด A:** สำหรับคะแนนตั้งแต่ 80 ขึ้นไป
- **เกรด B:** สำหรับคะแนนอยู่ในช่วง 70 ถึง 79
- **เกรด C:** สำหรับคะแนนอยู่ในช่วง 60 ถึง 69
- **เกรด D:** สำหรับคะแนนอยู่ในช่วง 50 ถึง 59
- **เกรด F:** สำหรับคะแนนที่ต่ำกว่า 50

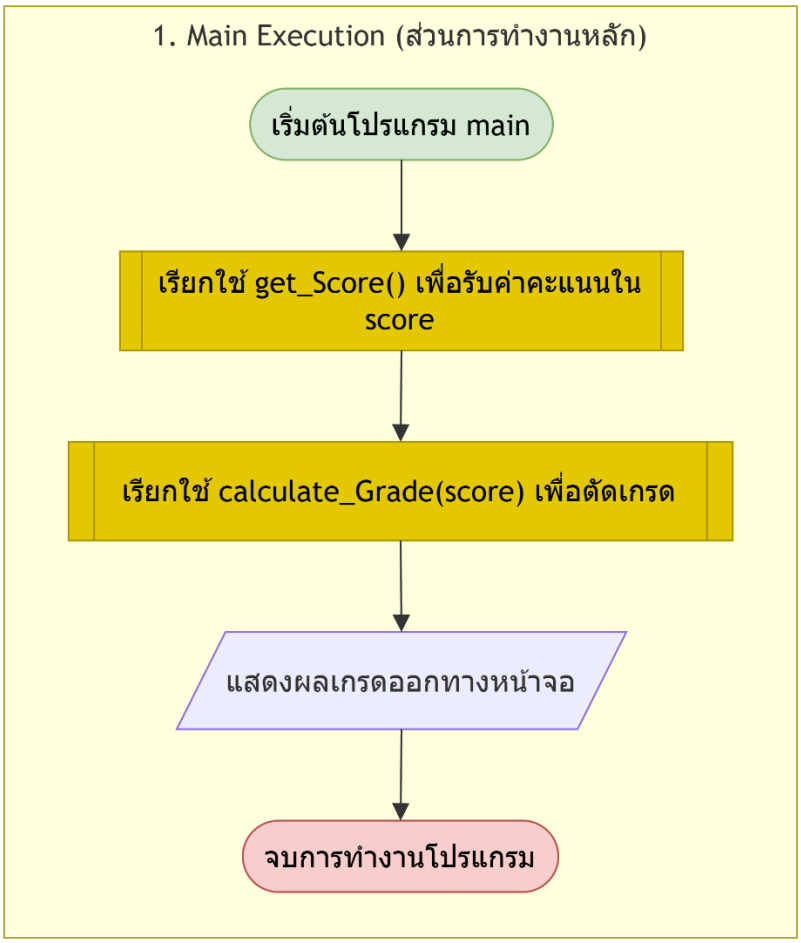
เมื่อเข้าเงื่อนไขใด ฟังก์ชันจะส่งคืนข้อความระดับเกรดนั้นๆ ออกมากระทบทันที

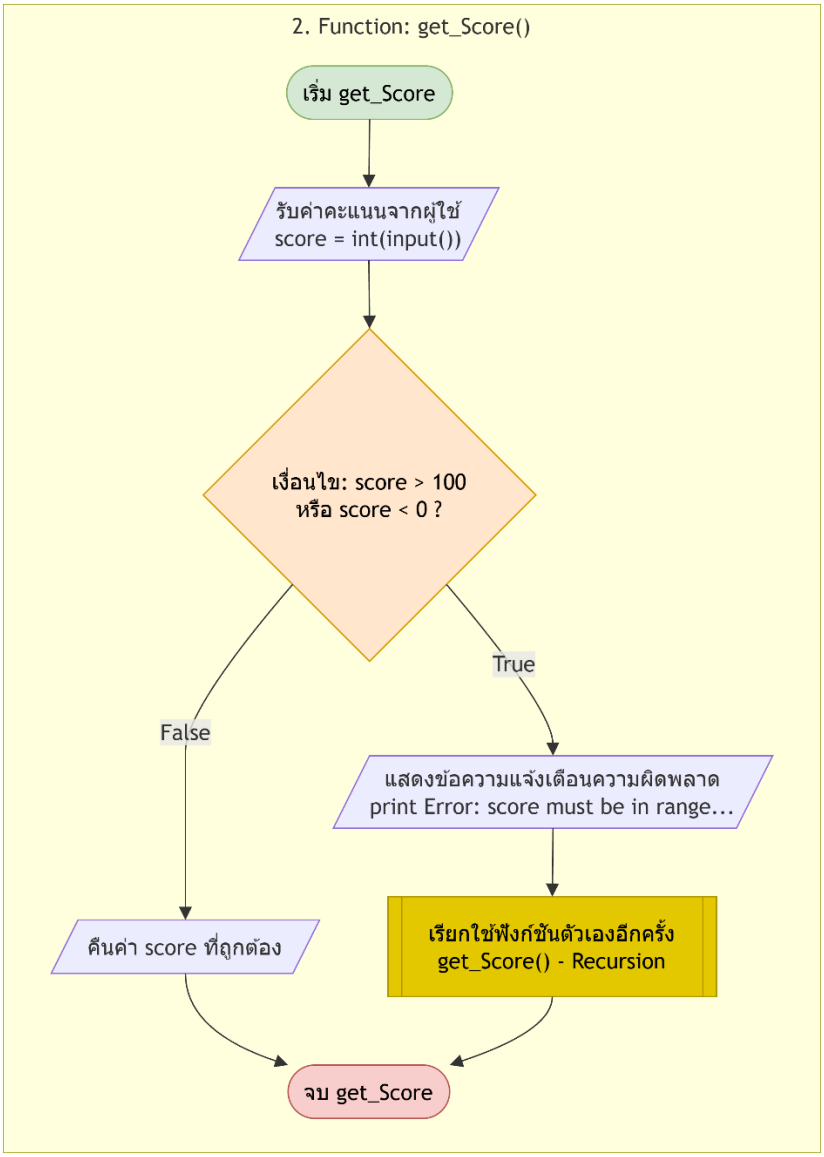
3. ฟังก์ชันรับข้อมูลและตรวจสอบความถูกต้อง

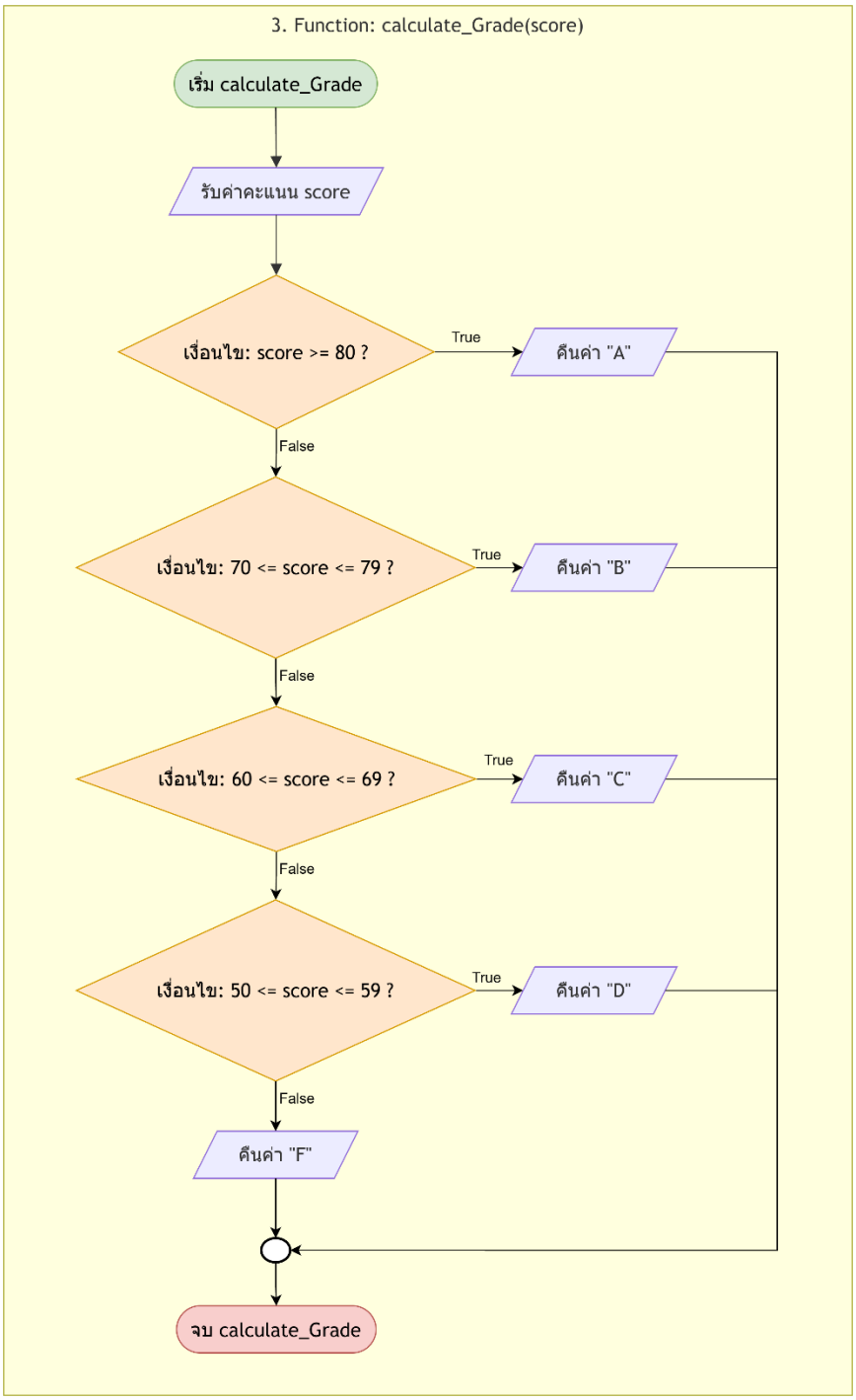
ฟังก์ชันส่วนนี้ทำหน้าที่ควบคุมความถูกต้องของการรับข้อมูลคะแนนจากผู้ใช้:

- **การรับค่าเข้า:** แสดงข้อความแจ้งเตือนให้ผู้ใช้ป้อนคะแนน แล้วทำการแปลงข้อมูลจากข้อความให้เป็นตัวเลขจำนวนเต็ม
- **การตรวจสอบเงื่อนไขความถูกต้อง:**
    - ตรวจสอบว่าคะแนนที่รับมามีค่าน้อยกว่า 0 หรือมากกว่า 100 หรือไม่
    - **กรณีป้อนข้อมูลผิดพลาด:** หากคะแนนไม่อยู่ในช่วง 0 ถึง 100 โปรแกรมจะแสดงข้อความแจ้งเตือนข้อผิดพลาดว่าคะแนนต้องอยู่ในช่วง 0 ถึง 100 เท่านั้น จากนั้นจะทำการ **เรียกใช้ฟังก์ชันตัวเองซ้ำ** เพื่อบังคับให้ผู้ใช้ป้อนคะแนนใหม่อีกครั้งจนกว่าจะถูกต้อง
    - **กรณีป้อนข้อมูลถูกต้อง:** ส่งคืนค่าคะแนนที่ถูกต้องกลับไปยังจุดเรียกใช้งาน

Flowchart


1. Main Execution (ส่วนการทำงานหลัก)
เริ่มต้นโปรแกรม main
เรียกใช้ get_Score() เพื่อรับค่าคะแนนใน score
เรียกใช้ calculate_Grade(score) เพื่อตัดเกรด
แสดงผลเกรดออกทางหน้าจอ
จบการทำงานโปรแกรม


2. Function: get_Score()
เริ่ม get_Score
รับค่าคะแนนจากผู้ใช้
score = int(input())
เงื่อนไข: score > 100
หรือ score < 0 ?
True
False
แสดงข้อความแจ้งเตือนความผิดพลาด
print Error: score must be in range...
เรียกใช้ฟังก์ชันตัวเองอีกครั้ง
get_Score() - Recursion
คืนค่า score ที่ถูกต้อง
จบ get_Score


3. Function: calculate_Grade(score)
เริ่ม calculate_Grade
รับค่าคะแนน score
เงื่อนไข: score >= 80 ?
True
คืนค่า "A"
False
เงื่อนไข: 70 <= score <= 79 ?
True
คืนค่า "B"
False
เงื่อนไข: 60 <= score <= 69 ?
True
คืนค่า "C"
False
เงื่อนไข: 50 <= score <= 59 ?
True
คืนค่า "D"
False
คืนค่า "F"
จบ calculate_Grade

Code

```python
def calculate_Grade(score):
    if score >= 80:
        return "A"
    elif score >= 70 and score <= 79:
        return "B"
    elif score >= 60 and score <= 69:
        return "C"
    elif score >= 50 and score <= 59:
        return "D"
    else:
        return "F"

def get_Score():
    score = int(input("Enter your score: "))
    if score > 100 or score < 0:
        print("Error: score must be in range from 0 to 100.")
        get_Score()
    else:
        return score

if __name__ == '__main__':
    score = get_Score()
    print(calculate_Grade(score))
```

```
Enter your score: 100
A
```

```
Enter your score: 79
B
```

```
Enter your score: 60
C
```

```
Enter your score: 50
D
```

```
Enter your score: 49
F
```

ข้อที่ 3

1. ภาพรวมของโปรแกรม

โปรแกรมนี้ทำหน้าที่คำนวณหาพื้นที่ของผืนดินทรงสี่เหลี่ยมผืนผ้าจากค่าน้ำหนักความกว้างและความยาวที่ผู้ใช้ป้อนเข้ามา จากนั้นจะนำขนาดของที่ดินดังกล่าวไปตรวจสอบกับขนาดมาตรฐานของสนามกีฬาประเภทต่างๆ (วอลเลย์บอล, บาสเกตบอล และฟุตบอล) ว่ามีขนาดเพียงพอที่จะจัดวางสนามเหล่านั้นลงไปได้หรือไม่ โดยรองรับการหมุนวางสนามทั้งแนวตั้งและแนวนอน

2. ฟังก์ชันคำนวณพื้นที่ดิน

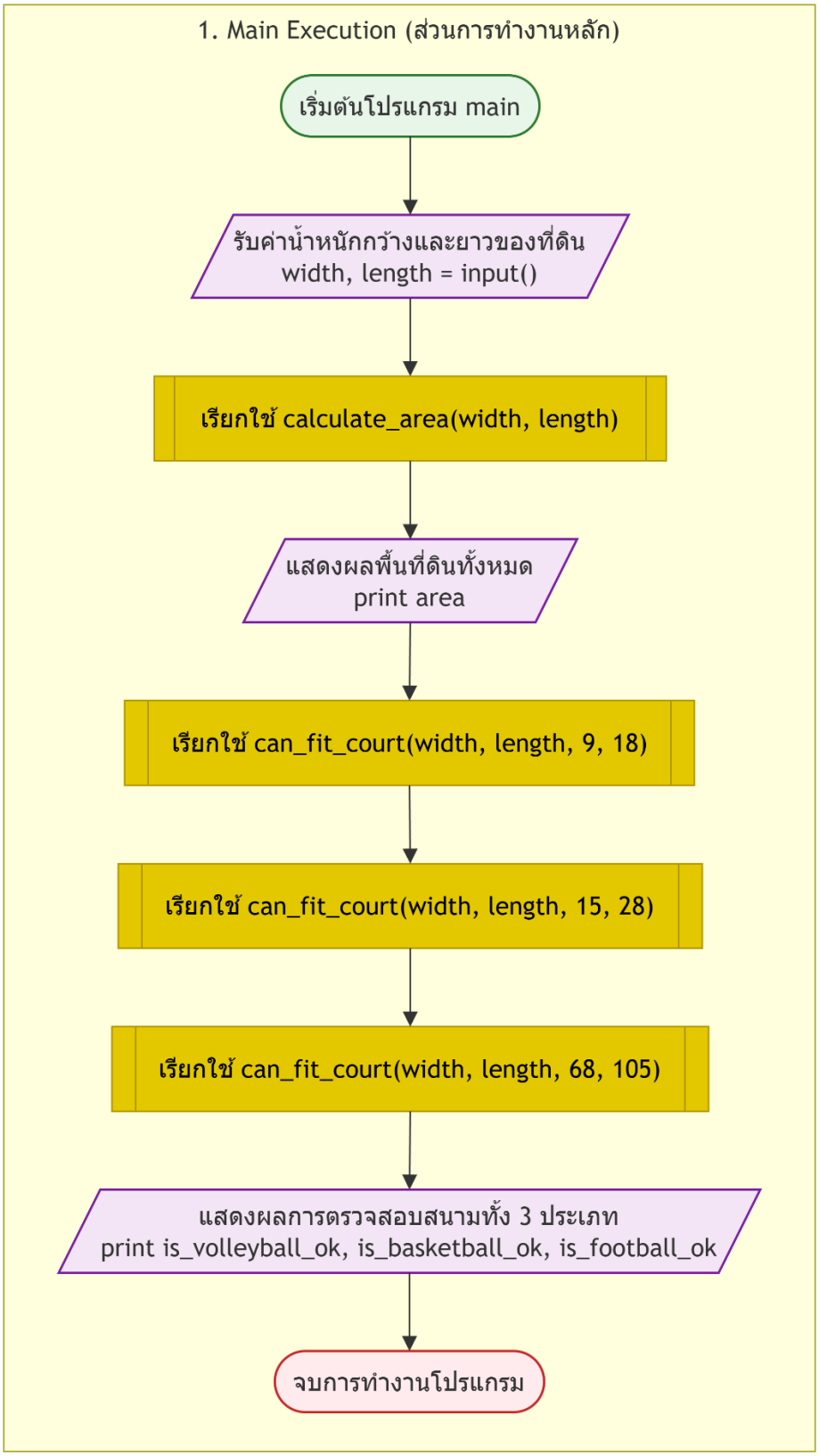
ฟังก์ชันส่วนนี้ทำหน้าที่รับข้อมูลขนาดความกว้างและความยาวของที่ดิน (ในหน่วยเมตร) เข้ามาประมวลผล โดยนำค่าทั้งสองมาคูณกันตามสูตรการหาพื้นที่รูปสี่เหลี่ยม และส่งคืนผลลัพธ์เป็นค่าพื้นที่รวมในหน่วยตารางเมตรกลับไปให้โปรแกรมหลัก

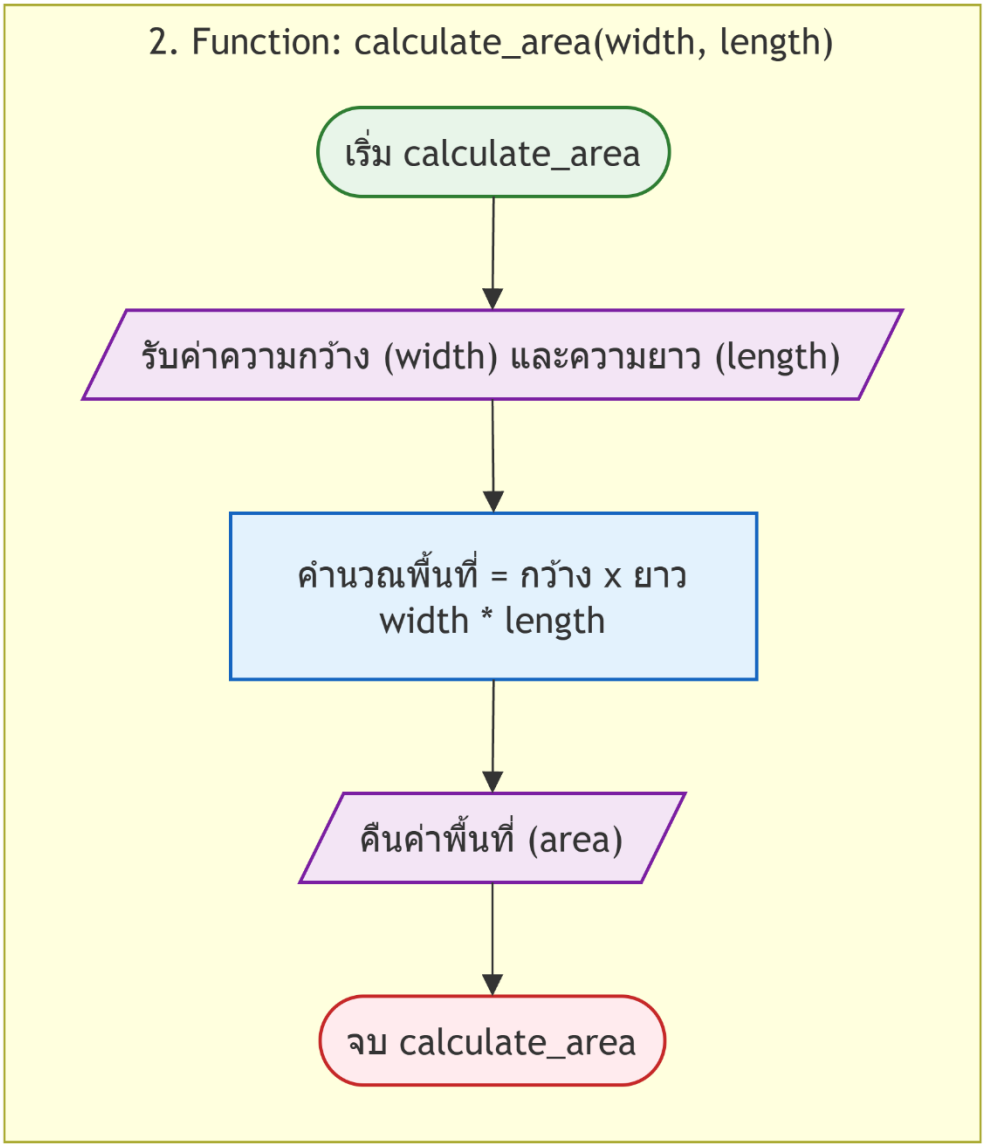
3. ฟังก์ชันตรวจสอบการจัดวางสนาม (Boolean Function)

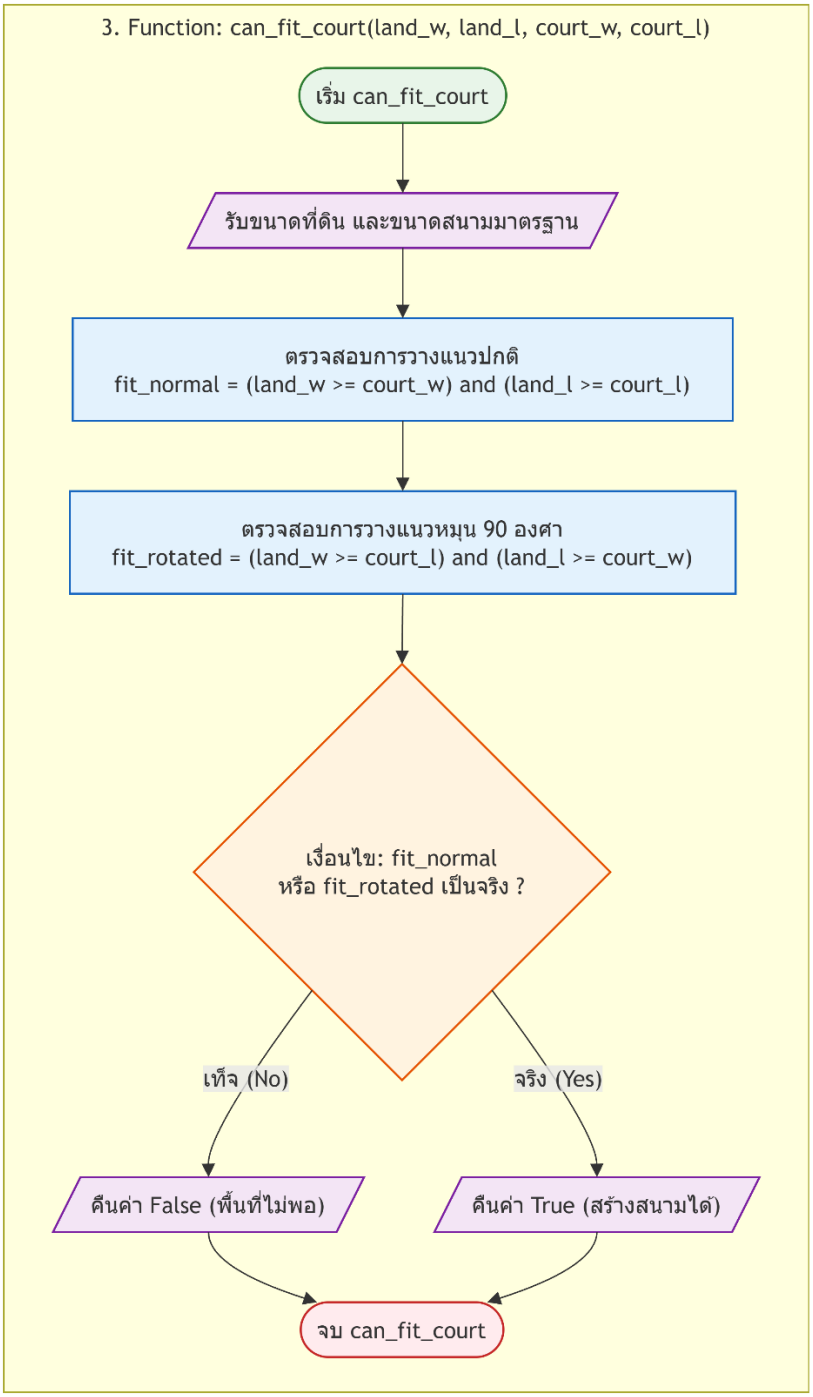
ฟังก์ชันส่วนนี้ถูกออกแบบเป็น **ฟังก์ชันคืนค่าความจริง (Boolean Function)** ทำหน้าที่ตรวจสอบว่าขนาดที่ดินสามารถสร้างสนามกีฬานั้นๆ ได้หรือไม่ โดยรับขนาดความกว้างและความยาวของทั้งที่ดินและสนามเข้ามาเปรียบเทียบ ผ่านตรรกะการตรวจสอบ 2 กรณี:

- **การวางสนามแนวปกติ:** ตรวจสอบว่าความกว้างของที่ดินมากกว่าหรือเท่ากับความกว้างของสนาม *และ* ความยาวของที่ดินมากกว่าหรือเท่ากับความยาวของสนามหรือไม่
- **การวางสนามแนวหมุน 90 องศา:** ตรวจสอบว่าความกว้างของที่ดินมากกว่าหรือเท่ากับความยาวของสนาม *และ* ความยาวของที่ดินมากกว่าหรือเท่ากับความกว้างของสนามหรือไม่
- **การสรุปผล:** นำผลการตรวจสอบทั้งสองกรณีมารวมกันด้วยตัวดำเนินการทางตรรกศาสตร์แบบ **OR (หรือ)** หากที่ดินวางสนามได้ไม่ว่าจะแนวใดแนวหนึ่ง ฟังก์ชันจะส่งคืนค่าความจริงเป็น **True (รองรับได้)** แต่ถ้าไม่สามารถวางได้เลยทั้งสองแนว จะส่งคืนค่า **False (พื้นที่ไม่พอ)**

Flowchart


1. Main Execution (ส่วนการทำงานหลัก)
เริ่มต้นโปรแกรม main
รับค่าน้ำหนักกว้างและยาวของที่ดิน
width, length = input()
เรียกใช้ calculate_area(width, length)
แสดงผลพื้นที่ดินทั้งหมด
print area
เรียกใช้ can_fit_court(width, length, 9, 18)
เรียกใช้ can_fit_court(width, length, 15, 28)
เรียกใช้ can_fit_court(width, length, 68, 105)
แสดงผลการตรวจสอบสนามทั้ง 3 ประเภท
print is_volleyball_ok, is_basketball_ok, is_football_ok
จบการทำงานโปรแกรม


2. Function: calculate_area(width, length)
เริ่ม calculate_area
รับค่าความกว้าง (width) และความยาว (length)
คำนวณพื้นที่ = กว้าง x ยาว
width * length
คืนค่าพื้นที่ (area)
จบ calculate_area


3. Function: can_fit_court(land_w, land_l, court_w, court_l)
เริ่ม can_fit_court
รับขนาดที่ดิน และขนาดสนามมาตรฐาน
ตรวจสอบการวางแนวปกติ
fit_normal = (land_w >= court_w) and (land_l >= court_l)
ตรวจสอบการวางแนวหมุน 90 องศา
fit_rotated = (land_w >= court_l) and (land_l >= court_w)
เงื่อนไข: fit_normal
หรือ fit_rotated เป็นจริง ?
เท็จ (No)
จริง (Yes)
คืนค่า False (พื้นที่ไม่พอ)
คืนค่า True (สร้างสนามได้)
จบ can_fit_court

Code

D: > KMUTNB > 1-69 > Computer-Programming > week3 > Lab > Lab3-06.py > ...

```python
def calculate_area(width, length):
    return width * length

def can_fit_court(land_width, land_length, court_width, court_length):
    fit_normal = (land_width >= court_width) and (land_length >= court_length)
    fit_rotated = (land_width >= court_length) and (land_length >= court_width)
    return fit_normal or fit_rotated

#รับข้อมูลความกว้างและความยาวจากผู้ใช้ (เมตร)
print("=== โปรแกรมคำนวณพื้นที่ดินและตรวจสอบสนามกีฬา ===")
width = float(input("ป้อนความกว้างของที่ดิน (เมตร): "))
length = float(input("ป้อนความยาวของที่ดิน (เมตร): "))

#คำนวณและแสดงผลพื้นที่
area = calculate_area(width, length)
print(f"\nพื้นที่ดินทั้งหมด: {area:,.2f} ตารางเมตร")
print("-------------------------------------------------------")

#ขนาดสนามมาตรฐาน (กว้าง x ยาว):
#Volleyball (9x18 เมตร), Basketball (15x28 เมตร), Football (68x105 เมตร)

is_volleyball_ok = can_fit_court(width, length, 9, 18)
is_basketball_ok = can_fit_court(width, length, 15, 28)
is_football_ok = can_fit_court(width, length, 68, 105)

print("ผลการตรวจสอบพื้นที่สำหรับสนามกีฬา:")
print(f"- Volleyball (วอลเลย์บอล): {is_volleyball_ok}")
print(f"- Basketball (บาสเกตบอล): {is_basketball_ok}")
print(f"- Football (ฟุตบอล): {is_football_ok}")
```

```
=== โปรแกรมคำนวณพื้นที่ดินและตรวจสอบสนามกีฬา ===
ป้อนความกว้างของที่ดิน (เมตร): 10
ป้อนความยาวของที่ดิน (เมตร): 19

พื้นที่ดินทั้งหมด: 190.00 ตารางเมตร
-------------------------------------------------------
ผลการตรวจสอบพื้นที่สำหรับสนามกีฬา:
- Volleyball (วอลเลย์บอล): True
- Basketball (บาสเกตบอล): False
- Football (ฟุตบอล): False
=== โปรแกรมคำนวณพื้นที่ดินและตรวจสอบสนามกีฬา ===
ป้อนความกว้างของที่ดิน (เมตร): 100
ป้อนความยาวของที่ดิน (เมตร): 200

พื้นที่ดินทั้งหมด: 20,000.00 ตารางเมตร
-------------------------------------------------------
ผลการตรวจสอบพื้นที่สำหรับสนามกีฬา:
- Volleyball (วอลเลย์บอล): True
- Basketball (บาสเกตบอล): True
- Football (ฟุตบอล): True
```

ข้อที่ 4

1. ภาพรวมของโปรแกรม

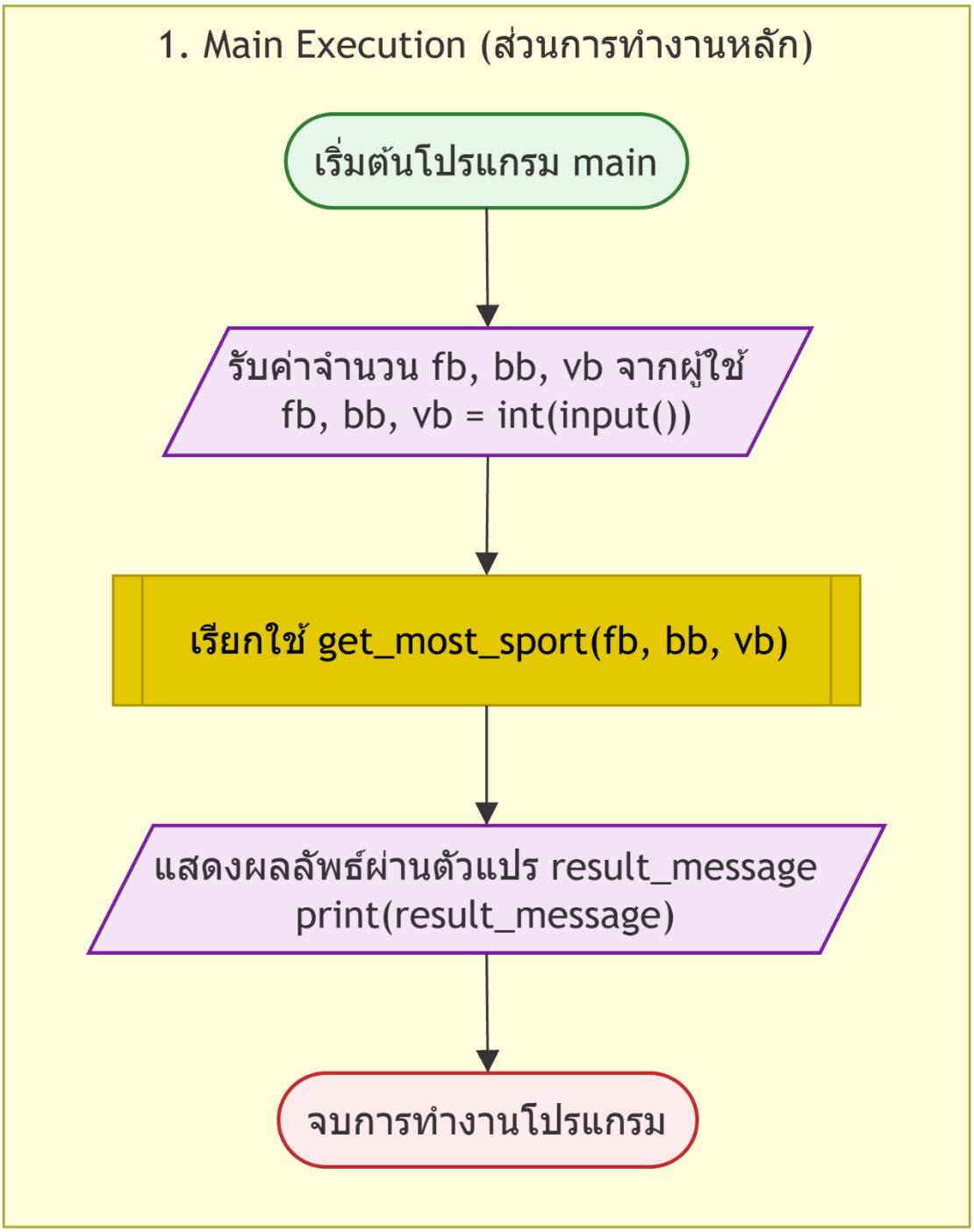
โปรแกรมนี้ทำหน้าที่รับค่าจำนวนลูกบอลของกีฬา 3 ประเภท (ฟุตบอล, บาสเกตบอล และวอลเลย์บอล) ในรูปแบบตัวเลขจำนวนเต็ม จากนั้นนำมาทำการเปรียบเทียบเชิงตรรกศาสตร์ด้วยคำสั่งเงื่อนไขพื้นฐาน เพื่อหาว่ากีฬาประเภทใดมีจำนวนมากที่สุด โดยถูกออกแบบมาให้รองรับทั้งกรณีที่มีกีฬาชนิดเดียวครองตำแหน่งมากที่สุด และกรณีที่มีกีฬาหลายชนิดมีจำนวนมากที่สุดเท่ากัน

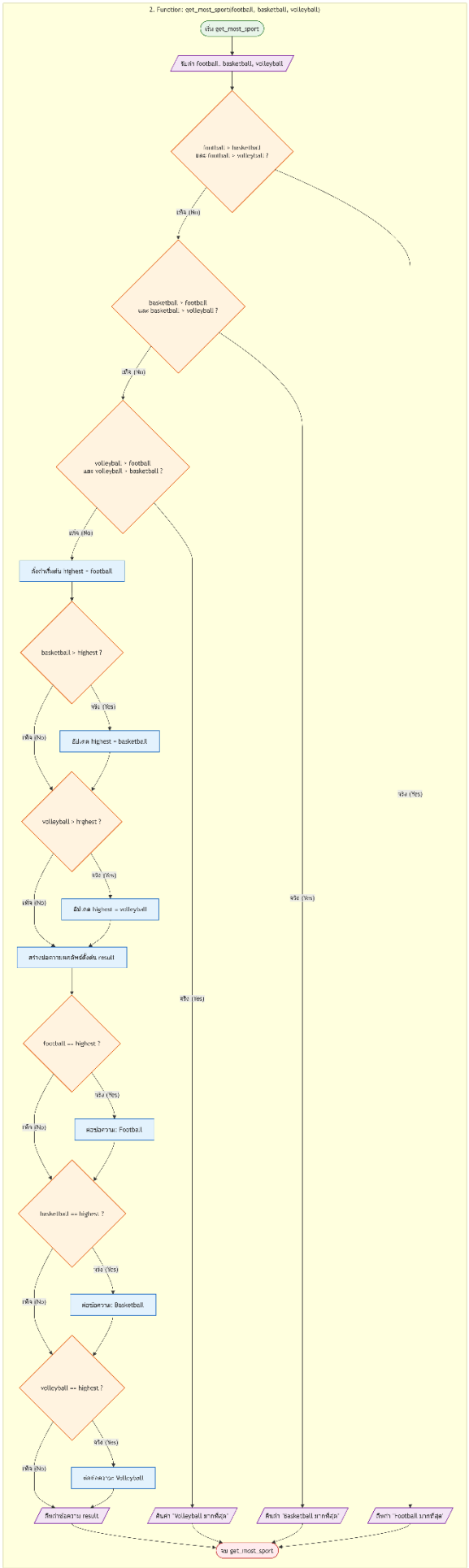
2. ฟังก์ชันเปรียบเทียบและค้นหากีฬาที่มีจำนวนมากที่สุด

ฟังก์ชันส่วนนี้รับค่าจำนวนลูกกีฬา 3 ค่าเข้ามาประมวลผล โดยแบ่งโครงสร้างการเปรียบเทียบออกเป็น 2 กรณีหลักผ่านคำสั่งทางเลือก:

- **กรณีมีกีฬาประเภทเดียวที่มากที่สุดเดี่ยวๆ**
    - ใช้ตัวดำเนินการทางตรรกศาสตร์ **AND** เข้ามาช่วยเปรียบเทียบแบบเงื่อนไขคู่ เพื่อเช็กว่ากีฬาชนิดใดชนิดหนึ่งมีจำนวนมากกว่าอีกสองชนิดที่เหลือพร้อมกันหรือไม่
    - หากตรงตามเงื่อนไข จะส่งคืนข้อความสรุปชื่อกีฬาชนิดนั้นๆ พร้อมระบุจำนวนลูกกลับออกไปทันที
- **กรณีที่มีจำนวนมากที่สุดเท่ากันมากกว่า 1 ประเภท**
    - หากไม่เข้าเงื่อนไขแรก (แปลว่าต้องมีอย่างน้อยสองประเภทที่จำนวนสูงสุดเท่ากัน) ระบบจะทำงานในส่วนการหาค่าสูงสุดก่อน โดยสมมติให้จำนวนลูกฟุตบอลเป็นค่ามากที่สุดเริ่มต้น จากนั้นนำจำนวนบาสเกตบอลและวอลเลย์บอลมาเปรียบเทียบทีละลำดับเพื่ออัปเดตค่าตัวเลขที่มากที่สุดจริง
    - เมื่อได้ค่าตัวเลขสูงสุดแล้ว โปรแกรมจะทำการตรวจสอบจำแนกทีละกีฬาอีกครั้งว่า กีฬาประเภทใดบ้างที่มีจำนวนเท่ากับค่าสูงสุดดังกล่าว แล้วนำรายชื่อกีฬาทั้งหมดที่เสมอกันมาต่อเป็นข้อความรายการขึ้นบรรทัดใหม่ เพื่อส่งคืนกลับไปแสดงผล

Flowchart


1. Main Execution (ส่วนการทำงานหลัก)
เริ่มต้นโปรแกรม main
รับค่าจำนวน fb, bb, vb จากผู้ใช้
fb, bb, vb = int(input())
เรียกใช้ get_most_sport(fb, bb, vb)
แสดงผลลัพธ์ผ่านตัวแปร result_message
print(result_message)
จบการทำงานโปรแกรม


2. Function: get_most_sport(football, basketball, volleyball)
เริ่ม get_most_sport
รับค่า football, basketball, volleyball
football > basketball
และ football > volleyball ?
เท็จ (No)
basketball > football
และ basketball > volleyball ?
เท็จ (No)
volleyball > football
และ volleyball > basketball ?
เท็จ (No)
ตั้งค่าเริ่มต้น highest = football
basketball > highest ?
จริง (Yes)
เท็จ (No)
อัปเดต highest = basketball
volleyball > highest ?
จริง (Yes)
เท็จ (No)
อัปเดต highest = volleyball
สร้างข้อความและตัวแปรสำหรับเก็บ result
จริง (Yes)
จริง (Yes)
จริง (Yes)
football == highest ?
จริง (Yes)
เท็จ (No)
ต่อข้อความ: Football
basketball == highest ?
จริง (Yes)
เท็จ (No)
ต่อข้อความ: Basketball
volleyball == highest ?
จริง (Yes)
เท็จ (No)
ต่อข้อความ: Volleyball
คืนค่าข้อความ result
คืนค่า "Volleyball มากที่สุด"
คืนค่า "basketball มากที่สุด"
คืนค่า "football มากที่สุด"
จบ get_most_sport

Code

```python
# ฟังก์ชันสำหรับเปรียบเทียบและ return ผลลัพธ์
def get_most_sport(football, basketball, volleyball):
    # กรณีที่มีอย่างใดอย่างหนึ่งมากที่สุดตัวเดียว
    if football > basketball and football > volleyball:
        return f"Football มากที่สุด ({football} ลูก)"
    elif basketball > football and basketball > volleyball:
        return f"Basketball มากที่สุด ({basketball} ลูก)"
    elif volleyball > football and volleyball > basketball:
        return f"Volleyball มากที่สุด ({volleyball} ลูก)"

    # กรณีที่มีจำนวนสูงสุดเท่ากันมากกว่า 1 กีฬา
    else:
        highest = football
        if basketball > highest:
            highest = basketball
        if volleyball > highest:
            highest = volleyball

        result = f"มีจำนวนมากที่สุดเท่ากัน ({highest} ลูก) ได้แก่:"
        if football == highest:
            result += "\n- Football"
        if basketball == highest:
            result += "\n- Basketball"
        if volleyball == highest:
            result += "\n- Volleyball"

        return result

#1. รับค่าตัวแปร integer 3 ตัว
fb = int(input("ป้อนจำนวนลูก Football: "))
bb = int(input("ป้อนจำนวนลูก Basketball: "))
vb = int(input("ป้อนจำนวนลูก Volleyball: "))

print("----------------------------------------")

#2. เรียกใช้ฟังก์ชัน แล้วรับค่าที่ return กลับมาเก็บไว้ในตัวแปร
result_message = get_most_sport(fb, bb, vb)

# 3. แสดงผลลัพธ์
print(result_message)
```

```
ป้อนจำนวนลูก Football: 1
ป้อนจำนวนลูก Basketball: 2
ป้อนจำนวนลูก Volleyball: 3
----------------------------------------
Volleyball มากที่สุด (3 ลูก)
 ป้อนจำนวนลูก Football: 2
 ป้อนจำนวนลูก Basketball: 2
 ป้อนจำนวนลูก Volleyball: 1
 ----------------------------------------
 มีจำนวนมากที่สุดเท่ากัน (2 ลูก) ได้แก่:
 - Football
 - Basketball
```